KLASY

Możliwość definiowania nowych typów danych
Klasa jako wzorzec do tworzenia obiektów
Enkapsulacja danych i operacji na składnikach

```
class name
{
private:
    data members;    //to prevent direct access, default
public:
    data members;    //available to objects
private:
    data members;    //only my family can use it
};                   //must have ;
```

Uwagi:

Deklaracja klasy nie tworzy obiektów!!!
Składniki nie są inicjowane w deklaracji klasy (wyjątkiem są składniki typu static const int wspólnie dla wszystkich obiektów klasy)
Możliwość deklaracji w obrębie funkcji
Powinno się deklarować na początku
Zwykle treść funkcji powinna być jednorazowa, jedna kopia funkcji składowych, aby nie multiplikować
Dostęp (jeden z możliwych):
Obiekt.składnik

Przykład deklaracji klasy:

```
class Address
{
    string street, city, country; //private by def
    int houseNumber, zipCode[6];
public:
    void display();               //shared
    int getNumber() const {       //inline function, can't change anything in private
        return houseNumber;
    }
};
```

Prywatnie mamy dane o adresie, publicznie używamy deklaracji funkcji display, która będzie dostępna dla każdego typu Adress, obiekty ją współdzielą. Jeżeli nie chcemy multiplikować kodu to używamy tego.

Funkcja inline to definicja, która każdemu obiektowi daje kopię funkcji.

Obie funkcje, display i getNumber, która pobrać numer domu z private, ale nie modyfikować.

Funkcja inline jest szybsza, ale display mniej zasobożerna. Time vs memory, trzeba na to patrzeć, jeżeli inline za długa to trzeba stosować tę pierwszą (współdzielona).

Definicja funkcji współdzielonej display();

```
void Address::display()      //:: scope operator
{
    cout << street << " " << house number << ... ;
}
```

Wywołanie wartości z klasy:

```
Address myAddress, univerity, myGirlFriend;      //summon 3 objects Address class!
int where = myGirlFriend.getNumber();            //using inline function getNumber();
university.display();                            //using shared function
```

TYPOWY BŁĄD! Sięgnięcie do private:

```
int nr = myAddress.houseNumber;
```

Po definicji klasy otrzymujemy w gratisie kilka dodatkowych elementów, niewidocznych, ale w które klasa jest wyposażona. Trzeba się z nimi liczyć. Ich niezamierzone uruchomienie może wywalić program na plecy.

KONSTRUKTOR

Konstruktor jest kolejną funkcją, którą każda klasa jest wyposażona. Jest to funkcja, która automatycznie jest uruchamiana kiedy obiekt jest tworzony. Jego zadania to:

- Inicjacja składników klasy
- Powoływanie wszystkiego w momencie tworzenia obiektu

Dobrze napisany program w C++ to taki, w którym my sami tworzymy treść konstruktora. PISZ ZAWSZE KONSTRUKTOR DO SWOJEJ KLASY!

```
Address::Address(int actualNumber):      //constructor name same as class name
    houseNumber(actualNumber)            //initialization list, nie obowiazkowe
    {...}                                //constructor body

Address myAddress(12);                   //here constructor start
```

Ostatnia linia - utwórz obiekt myAddress, z parametrem actualNumber==12, przyporządkuj houseNumber==12.

Przeładowanie (overflow): możliwa definicja wielu konstruktorów:

```
Address::Address(int actualNumber):     //constructor name same as class name
    houseNumber(actualNumber)           //initialization list, nie obowiazkowe
    {...}                               //constructor body

Address::Address(string strName):      //constructor name same as class name
    street(strName)                    //initialization list, nie obowiazkowe
    {...}                              //constructor body
```

Wywołanie różnych konstruktorów:

```
Address myAddress(12);
Address myAddress(Polanka);
```

Konstruktor delegujący to taki, który wywołuje inny konstruktor z tej samej klasy na liście inicjującej.

Duży przykład:

```
class Circle
{
    double radius;
public:
    Circle(double r):       //constructor start
        radius(r)
        {

        }                   //constructor end
    double area()           //inline function
    {
        return 3.14*raius*radius;
    }
};

class Cylinder
{
    double height; Circle base; //object of class Circle
public:
    Cylinder(double h, double r):
        height(h), base(r)      //call CIRCLE constructor, using Circle object
        {

        }
    double volume()
    {
        return height*base.area();
    }
};
```

Najlepiej jednak konstruktor robić globalnie, poza definicją klasy.

Wywołanie tego:

```
int main()
{
    Cylinder column(10,20); //make Cylinder object, that calls cylinder constructor, that calls circle constructor
    return 0;
}
```

DESTRUKTOR

Oczywiście lepiej napisać go jawnie samemu! Funkcja, która z definicji jest uruchamiana gdy przystępujemy do likwidacji obiektu.

```
Address::~Address()
{...}
```

Działamy jak mafia - zabijamy obiekt i całą rodzinę, automatyczny destruktor może oszczędzić rodzinę, na co nie można pozwolić.
Destruktora nie możemy przeładować.

LICZBY ZESOPLONE

Klasa dla liczb zespolonych:

```
int main() {
    Cylinder column(10, 20);
    ...
    return 0;
}
```

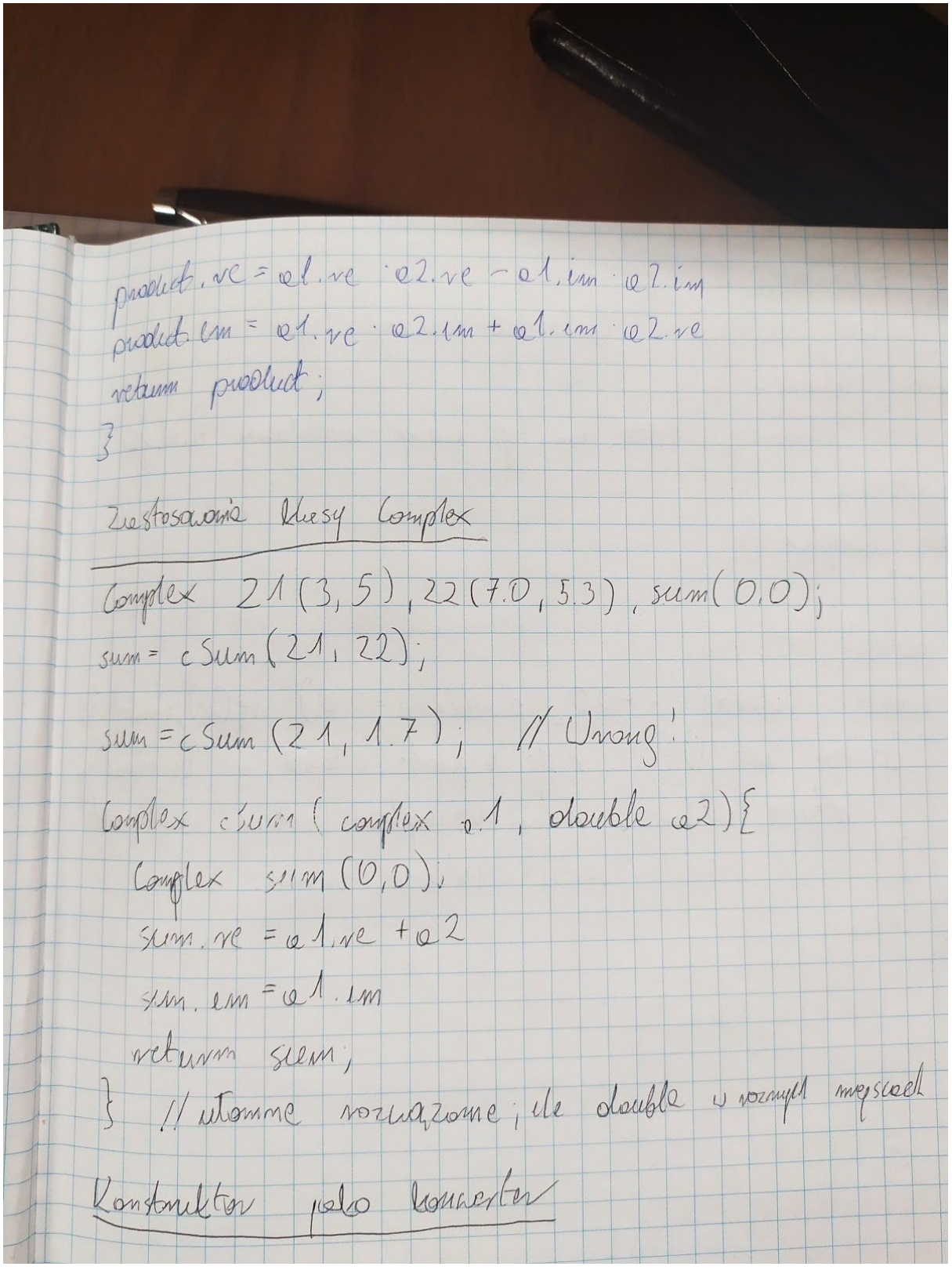
Destruktor - funkcja wykonywana w chwili usuwania obiektu

```
Address :: ~Address() { ... }
```

Liczby zespolone

```
class Complex {
  public:
    double re, im;
    Complex (double r, double i) : re(r), im(i) {}
};

Complex cSum (Complex c1, Complex c2){
  Complex sum(0, 0);
  sum.re = c1.re + c2.re;
  sum.im = c1.im + c2.im;
  return sum;
}
Complex cMult (Complex c1, Complex c2 ){
  Complex product (0,0);
```



```
product.re = c1.re * c2.re - c1.im * c2.im
product.im = c1.re * c2.im + c1.im * c2.re
return product;
}
```

Zastosowanie klasy Complex

```
Complex c1(3,5), c2(7.0, 5.3), sum(0,0);
sum = cSum(c1, c2);

sum = cSum(c1, 1.7);   // Wrong!

Complex cSum(complex c1, double c2){
  Complex sum(0,0);
  sum.re = c1.re + c2
  sum.im = c1.im
  return sum;
} // wtorne rozwiązanie; ale double w rozmyśl mojszeli
```

Konstruktor jako konwerter

Wywołanie klasy i funkcji cSum:

```
1  Complex Z1(3,5), Z2(7.0,5.3), sum(0,0);
2  sum=cSum(Z1,Z2);
3
4  sum=cSum(Z1,1.7); //WRONG!!
5
```

Błąd - funkcja cSum wymaga dwóch argumentów typu Complex.

W tym przypadku kompilator nie może dokonać niejawnego rzutowania, gdyż nie potrafi rzutować do nowego, stworzonego dopiero typu Complex.

Rozwiązanie wygląda następująco:

```
8  Complex cSum(complex a1, double a2)
9  {
10     Complez sum(0,0);
11     sum.re=a1.re+a2;
12     sum.im=a1.im;
13     return sum;
14 }
15
16 sum=cSum(1.7, Z2); //STILL WRONG!!
```

Kolejny błąd - ważna jest kolejność, musi zostać uwzględnione to w kolejnej definicji naprawczej. Jednak jest to rozwiązanie typu "duct tape".

Musimy poinstruować program co ma robić, by konwertować liczby double na liczby zespolone.

KONSTRUKTOR JAKO KONWERTER

Możemy liczbom nadać domyślne wartości:

```
19 class Complex
20 {
21     public:
22     double re, im;
23
24     Complex (double r=0.0, double r=0.0):
25     re(r), im(i)
26     {
27
28     }
29 }
```

Dzięki argumentom domniemanym konstruktora Complex zawiera konwersję double > Complex.

```
32 Complex argument(3.4, 6.9), result;
33 double second=2.5;
34 result=cSum(argument, Complex(second));
35
36
```

WIĘCEJ O KONWERSJI:

- Konstruktor umożliwia konwersję obcego typu na typ własnej klasy

- Konwersję własnego typu ja obcy umożliwia funkcja konwertująca
- Musimy wtedy wyposażyć klasę w tę funkcję

Tutaj funkcja konwertuje część rzeczywistą na typ int. Nigdy nie ma parametrów, bo jest unikalna.

```
38 class Complex
39 {
40     public:
41     double re, im;
42 
43     Complex (double r=0.0, double r=0.0):
44     re(r), im(i)
45     {
46 
47     }
48     operator int()
49     {
50         return int(re);
51     }
52 }
```

NIE DA SIĘ WYWOŁAĆ KONSTRUKTORA JAKO FUNKCJI! TYLKO KIEDY OBIEKT JEST TWORZONY! Destruktor za to można tak wywołać.

Tutaj jednak składniki są publiczne.

Mechnaizm, który umożliwia operowanie na składnikach znajdujących się w private to:

FUNKCJE ZAPRZYJAŹNIONE
Funkcja zaprzyjaźniona z klasą to taka, która ma dostęp do private. Jak kumpel wchodzący do domu i podlewający kwiatki.

```
1  Class B;
2  Class A
3  {
4      public:
5      int reader(B)
6  };
7 
8  Class B
9  {
10     int number;
11     friend int A::reader(B); //this function is
   friend
12 };
13 
14 int A::reader(B object)
15 {
16     return object.number;
17 }
```

WIĘCEJ O RRZYJAŹNI
- Można użyć FRIEND CLASS A, aby cała klasa A była przyjacielem klasy B.
- Przyjaciel mojego przyjaciela NIE JEST moim przyjacielem
  🅰️➡️♥️➡️🅱️➡️♥️➡️⬛ ALE ⬛➡️🖤➡️🅰️

- Przyjaźń nie jest zamienna

POWRÓT DO ZESPOLONYCH

Aby użyć w naszej funkcji

```
24 Complex cSum(Complex a1, Complex a2)
25 {
26     Complex sum;
27     sum.re=a1.re+a2.re;
28     sum.im=a1.im+a2.im;
29     return sum;
30 }
```

Składników re i im, gdy będą w private musimy dodać do public:

```
18
19
20 friend Complex cSum(Complex, Complex);
21
22
23
```

PRZEŁADOWANIE OPERATORÓW

Tak wygląda każda operacja, z każdym operatorem:

```
32 int C=A operator B;
33
34 return_type operatorΔ
   (parameter_declaration_list) //Δ dowolny
   operator
35 {
36     local_declatarions;
37     statement_list;
38     return expression;
39 }
```

Oznacza to, że możemy zredefiniować znaczenie każdego operatora (+,-,*,/).
Np. można sprawić, by + dodawał liczby zespolone.

- Nie zastępuje on niczego, a tylko dodaje nową opcję.
- Nie można zmienić priorytetów operatorów, reguł łączności oraz liczby argumentów.

Przeładowany operator musi używać co najmniej jednego typu stworzonego przez nas.
Typowy błąd:

```
41
42
43 operator+(int,double)
44
45
```

Przeładować można każdy występujący w C++ operator.

Np. przeładujmy +, aby dodawał liczby zespolone:

```
45
46 Complex operator+(Complex a1, Complex a2)
47 {
48     return Complex(a1.re+a2.re, a1.im+a2.im);
49 }
50
```

Co pozwala nam tworzyć takie działania:

```
52
53
54 Complex Z1(3,5), Z2(1.2,9.3), sum;
55 sum=Z1+Z2;
56
57
```

Musimy jednak dodać do klasy przyjaciela przeładowującego:

```
56
57
58 friend Complex operator+(Complex a1, Complex
   a2)
59
60
```

NA SZCZĘŚCIE są operatory, które są przeładowane z definicji, dlatego = może przypisać Complex do Complex.

Dlaczego jednak operatory i inne nie są częścią klasy? Bo wtedy mogą pobrać dane i elementy z całego programu. Gdyby był w klasie to jeden element musi być z tej klasy.

Reguła: jeżeli operator zmienia zawartość składników klasy to przeładowujemy w klasie. Przykładem są: =,++,+=,*=, etc.

- Uwaga: operatory przeładowane w klasie mają jeden argument mniej.
- Operatory predefiniowane dla każdej klasy: =, &, ,, new, delete.

Przykład:

```
64
65 Complex operator+(Complex);
66 };
67
68 Complex Complex::operator+(Complex a1)
69 {
70     return Complex(re+a1.re, im+a1.im);
71 }
72
73 double second;
74 Complex result=second+argument;
75 Complex result=second.operator(argument); //
   WRONG!!!!!
```

Argument jest tutaj dowolnym obiektem typu Complex.
Skutek przeładowania wewnątrz klasy - dodawanie nie jest przemienne. Więc lipa.

DZIEDZICZENIE
Definicja klasy pochodnej jest rozszerzana o składniki klasy bazowej

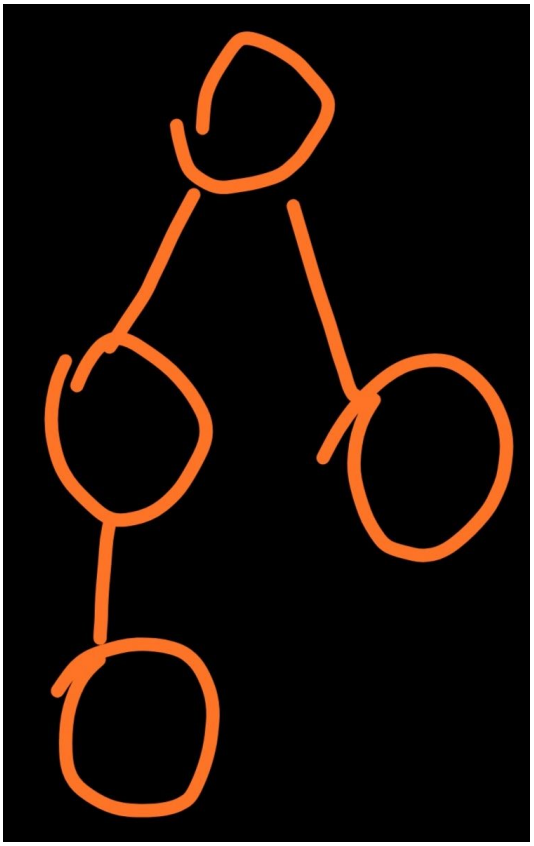
```
class new_class:public base_class
{
    class_definition;
};

```

Klasa bazowa - uogólnienie klas pochodnych
Klasy pochodne - specjalizacje klasy bazowej
Zwykle tworzy się obiekty klas pochodnych

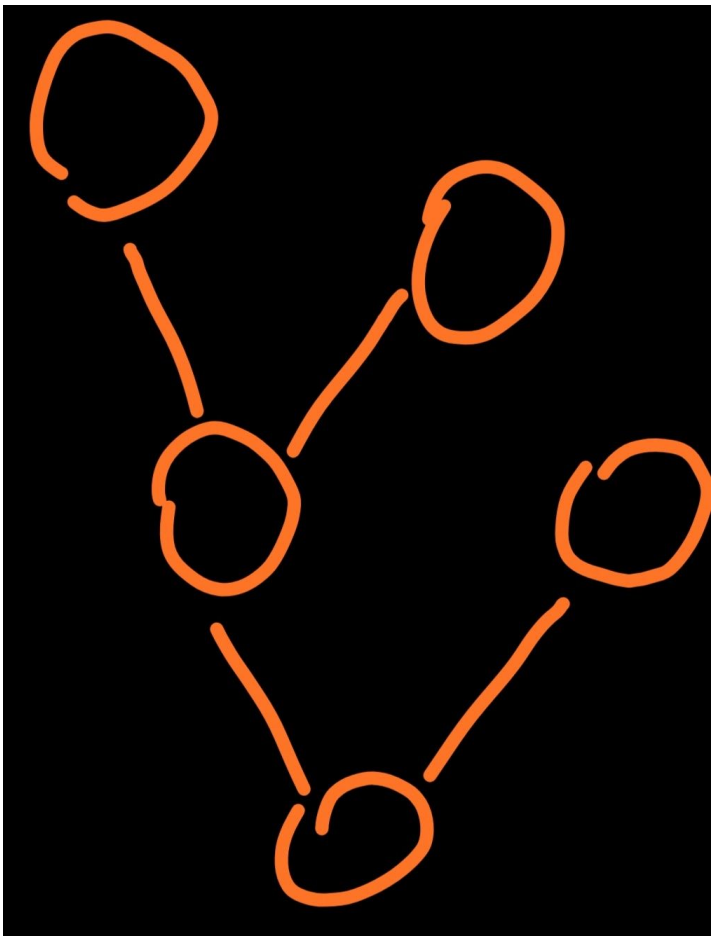
Zamiast **public base_class** można użyć **protected** lub **private**.

HIERARCHIA KLAS
Dziedziczenie pojedyncze: jedna klasa bazowa



Dobrze jest w ten sposób dokumentować.

Dziedziczenie wielokrotne: klasa pochodna jest związana z wieloma klasami bazowymi.



Przykład dziedziczeń:

```
class Vechicle
{
    private:
    double weight;
    int wheels;

    public:
    void()
    {
        cout << "Starting vechicle";
    }
};

class Car:public Vehicle
{
    public:
    void displayName();
    int seats;
}

class Motorcycle:public Vehicle, public Bike
{

};
```

DOSTĘP DO SKŁADNIKÓW

Klasa_pochodna:private klasa_bazowa:

- Private - niedostępne z klasy pochodnej
- Protected, public - jest przenoszone do części prywatnej klasy pochodnej

Klasa_pochodna:protected klasa_bazowa:

- Private - niedostępne z klasy pochodnej
- Protected, public - jest przenoszone do części protected klasy pochodnej

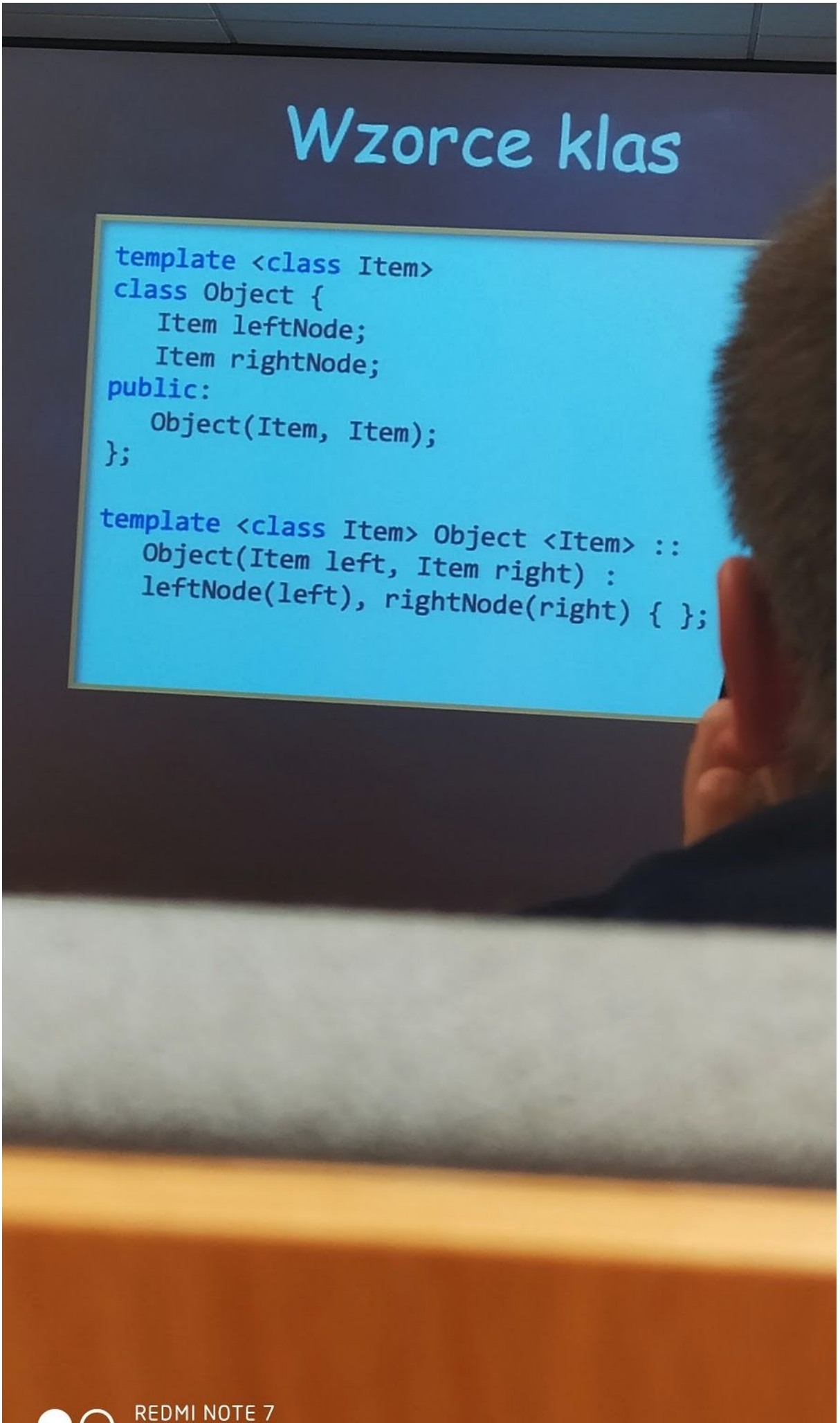
Klasa_pochodna:public klasa_bazowa:

- Private - niedostępne z klasy pochodnej
- Protected - jest przenoszone do części prywatnej klasy pochodnej
- Public - jest przenoszone do części publicznej klasy pochodnej

UWAGI O DZIEDZICZENIU:

- Podczas tworzenia obiektu konstruktory klas bazowych wywoływane są od klasy najstarszej w hierarchii (od bazy).
- Konstruktory wymagające argumentów występują na liście inicjacyjnej konstruktora klasy pochodnej.
- Składniki tak samo nazwane z różnych klas bazowych wymagają operatora okresu ::
- Przyjaźń nie jest dziedziczna - jeżeli klasa bazowa jest zaprzyjaźniona z inną, to klasa pochodna nie jest.
- Sekcja **protected** umożliwia dostęp funkcjom klasy pochodnej (i jej przyjaciołom) do "pół-prywatnych" składników klasy bazowej (pochodna może sięgać do sekcji **protected** bazowej, we wszystkim innym jest ona tak jak private).
- Kwalifikator **final** w nazwie klasy uniemożliwia wykorzystanie jej jako bazowej (od C++11) dodawane po nazwie klasy

WZORCE KLAS


Wzorce klas
template <class Item>
class Object {
Item leftNode;
Item rightNode;
public:
Object(Item, Item);
};
template <class Item> Object <Item> ::
Object(Item left, Item right) :
leftNode(left), rightNode(right) { };

REDMI NOTE 7

```
1 Object<int> integerPair(3, 5);
```

WSKAŹNIKI
Zmienna wskaźnikowa przechowuje adres obiektu danego typu

```
1
2
3 int *Pointer
```

Adres zmiennej udostępnia operator &

```
1 int *Pointer;
2
3 int number = 3;
4 Pointer = &number;
```

"Odczytaj adres zmiennej **number**, po czym podstaw adres pod **Pointer**"

```
1 int value = *Pointer;
```

"Weź z **Pointer** adres komórki, sprawdź co tam jest, po czym podstaw pod **value** wartość tej komórki."

OPERATORY NEW I DELETE
Dynamiczne tworzenie obiektów

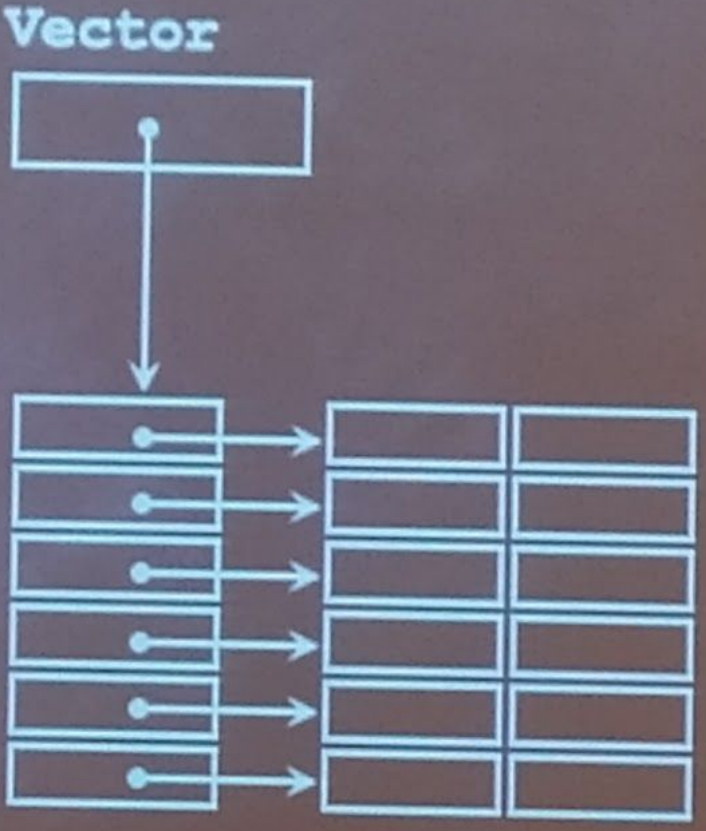
```
1 complex *number = new complex(2,3);
2
3
```

Takie obiekty nie mają nazwy i nie obowiązują ich zasady widzialności (jak globalne zmienne).

Obiekty istnieją do momentu ich usunięcia za pomocą operatora **delete**
Jeżeli jest mniej **delete** niż **new** pojawia się **memory leak**.

DYNAMICZNA ALOKACJA TABLICY
Nazwa tablicy adresem jej pierwszego elementu

szego elementu



REDMI NOTE 7

Pojedyncza tablica

```
1 //1-D array
2 cin >> size;
3 double *vector = new double[size];
4 vector[5] = 33; //same as below
5 *(vector+5) = 33; //same as above
6 vector[0] = 15;
7 *vector = 15;
8 delete [] vector;
```

***(vector+1)** oznacza "przejdź do kolejnej komórki tego elementu, przesuń się o jedno słowo binarne"

Dodatkowo

```
10 pointer = array //pointer = &array[0];
11
12 for(int i = 0; i < 3; i++)
13 {
14     *(pointer++) = -1;
15 }
```

Dwuwymiarowa tablica

```
16
17 //2-D array
18 double **vector;
19 cin >> size1;
20 cin >> size2;
21 vector = new double*[size1];
22 for(int i = 0; i < size1; i++)
23 {
24     vector[i] = new double[size2];
25 }
26
```

FUNKCJE WIRTUALNE I POLIMORFIZM

Wskaźnik, jako zmienna wskazująca na obiekty klasy może też służyć do wskazywania na obiekty jej klas pochodnych. Działa to tylko na pochodne, nie na bazowe (tylko w jedną stronę).

```
1  Vehicle bus;
2  bus.start();
3
4
5  class Car: public Vehicle
6  {
7      public:
8          void start()
9          {
10             cout << "Starting a car.";
11         }
12 };
13
14 Vehicle *p; //wskaźnik na obiekty klasy Vehicle
15 p = new Car; //nowy obiekt klasy Car, pochodnej Vehicle
16 p->start(); //wykonaj funkcje start() na obiekcie typu Car zapisanego we wskaźniku p
17
```

Poprzez arbitralną decyzję twórców C++ uruchomiona zostanie funkcja klasy bazowej!!!
Żeby uruchomić funkcję klasy pochodnej trzeba poprzedzić tę funkcję w klasie bazowej słowem VIRTUAL

```
19  class Vehicle
20  {
21      public:
22          virtual void start()
23          {
24              cout << "Starting a vehicle.";
25          }
26  };
27
28  p->start(); //wykonaj funkcje start() na obiekcie typu Car zapisanego we wskaźniku p
```

Tutaj wykona funkcję z klasy Car

- taka sama nazwa, taki sam typ wyniku i taka sama sygnatura
- definiowane w klasach bazowej i pochodnych
- wersja funkcji zależy od obiektu klasy użytego do wywołania
- w klasie bazowej ze słowem **virtual**, w klasach pochodnych można dodać **override**, aby sprawdzić przesłanianie funkcji z klasy bazowej
- kwalifikator **final** uniemożliwia dalsze przesłanianie funkcji z klasy bazowej w kolejnych klasach pochodnych
- czysta funkcja wirtualna - klasa abstrakcyjna, czysto kosmetyczny element, klasa jest abstrakcyjna, bo nie wolno nam powołać obiektów tej klasy, można powołać dopiero obiekty klas pochodnych, dziedziczącej niej:

```
31  virtual doube sum(int) = 0;
```

POJEMNIKI (ANG. CONTAINERS)

Obiekty zdolne do przechowywania innej klasy nazywamy pojemnikami. Zwykle definiowane za pomocą szablonów klas. Pojemniki dostępne w STL:

- wektory
- listy
- talie (listy wektorów)
- zbiory (w tym binarne)
- stosy
- kolejki i kolejki priorytetowe

Pojemnik typu wektor:

```
template<class T> class Vector
{
    protected:
        T *data; //data area
        unsigned int size;
    public:
        Vector(unsigned int noElements);
        Vector(unsigned int noElements, T & initial);
        Vector(const Vector & source); //copy constructor
        virtual ~Vector();

        T & operator [](unsigned int index) const; //access
        unsigned int lenght() const; //vector lenght
        //const - cannot change member variables unless "mutable"
        unsigned int setSize(unsigned int noElements); //change size
        unsigned int setSize(unsigned int noElements, T & initial);
};
```

```
template <class T> Vector <T> ::
    Vector(unsigned int noElements):
        size(noElements)
        {
            data = new T[size];
        }

template <class T> Vector <T> ::
    Vector(unsigned int noElements, T & initial):
        size(noElements)
        {
            data = new T[size];
            for(int i = 0; 1 < size; i++)
            {
                data[i] = initial; //set to initial value
            }
        }
```

```
3   template<class > class BoundedVector: public Vector<T>
4   {
5       protected:
6           const int lowBound;
7       public:
8           BoundedVector(int lowIndex, int highIndex);
9           BoundedVector(int lowIndex, int highIndex, T & initial);
10          BoundedVector(const BoundedVector & source);  //konstruktor kopiujacy
11
12          T & operator [](int index) const;
13          int lowerBound();
14          int upperBound();
15  }
```

Konstruktory:

```
18  template<class T> BoundedVector<T>::BoundedVector(int lowIndex, int highIndex):
19      Vector<T>(highIndex - lowIndex + 1), lowBound(lowIndex) //only way to initialize constant
20      {
21          assert(lowIndex <= highIndex);  //jako argument bierze relacje logiczna i sprawdza prawdziwość
22                                          //jeśli relacja jest nieprawdziwa, przerywa program
23      }
24
25  template<class T> BoundedVector<T>::BoundedVector(int lowIndex, int highIndex, T & initial):
26      Vector<T>(highIndex - lowIndex + 1, initial), lowBound(lowIndex) //only way to initialize constant
27      {
28          assert(lowIndex <= highIndex);  //jako argument bierze relacje logiczna i sprawdza prawdziwość
29                                          //jeśli relacja jest nieprawdziwa, przerywa program
30      }
```

Operator indexu:

```
32  template<class T>
33      T & BoundedVector<T> :: operator[](int index) const
34      {
35          return Vector<T> :: operator[](index - lowBound);
36      }
37
38  using namespace std;
```

Użycie operatora [ ] z klasy Vector (koniecznie desygnowanie dostępu dla uniknięcia rekurencyjnego wywołania operatora [ ] z klasy BoundedVector. Parametr zmienną całkowitą (możliwe użycie indeksu ujemnego).

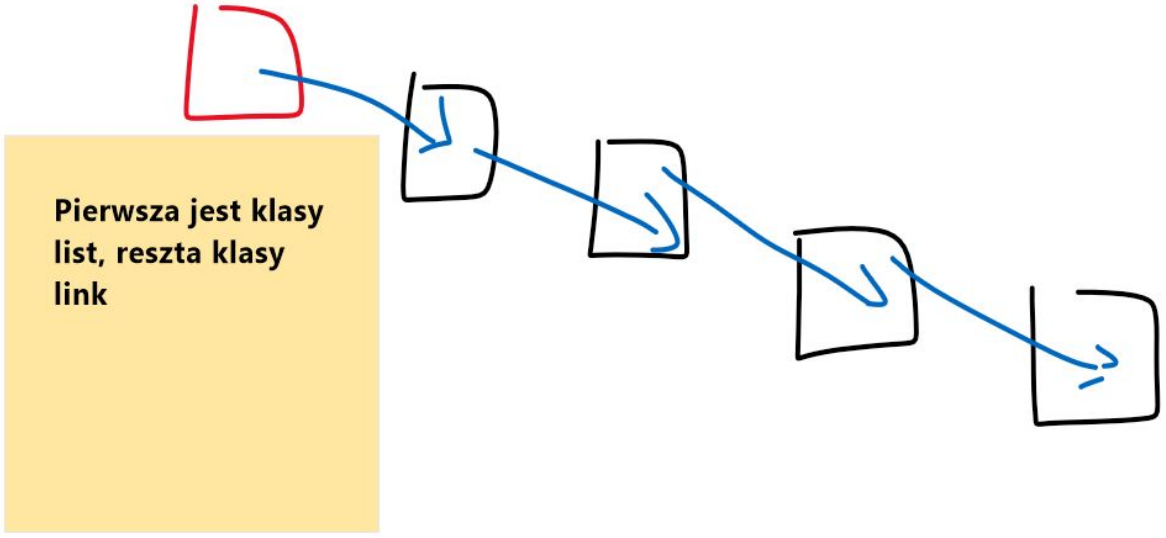
Pozostałe funkcje: zwracające elementy prywatne, jak loweBound czy upperBound.

BUDOWA HISTOGRAMU ZLICZAJĄCEGO LICZBĘ WYSTĄPIEŃ LITER ALFABETU W ANALIZOWANYM TEKŚCIE

```
41  void histogram()
42  {
43      int zero = 0;
44      BoundedVector<int> counts('a', 'z', zero);
45      string word;
46      while(cin >> word)
47      {
48          countLetters(counts, word);
49      }
50      for(char c = 'a', c <= 'z'; c++)
51      {
52          cout << "letter " << c << ": " << counts[c] << endl;
53      }
54  }
```

LISTY (JEDNOKIERUNKOWE)

Przy listach w przeciwieństwie do tablic kompilator mogą być alokowane w całej pamięci. Dostęp mamy tylko przez GŁOWĘ, czyli element zawierający wskaźnik na pierwszy element. Tutaj pojedziemy ostro ze wskaźnikami.



Zaczynamy od klasy List:

```
template<class T> class list
{
    protected:
            Link<T> *first; //wskaźnik wskazujący na pierwszy obiekt klasy Link, pierwszy element listy
                            //dzięki niemu mamy dostęp do listy
    public:
        List():
            first(nullptr) //nullptr to domyślny zerowy wskaźnik
        {

        }
        virtual ~List();

        virtual void add(T value); //schemat dodający jako pierwsze miejsce na liście
        friend class Iterator<T>;
}
```

To jest podstawowa, poniżej bardziej rozbudowana:

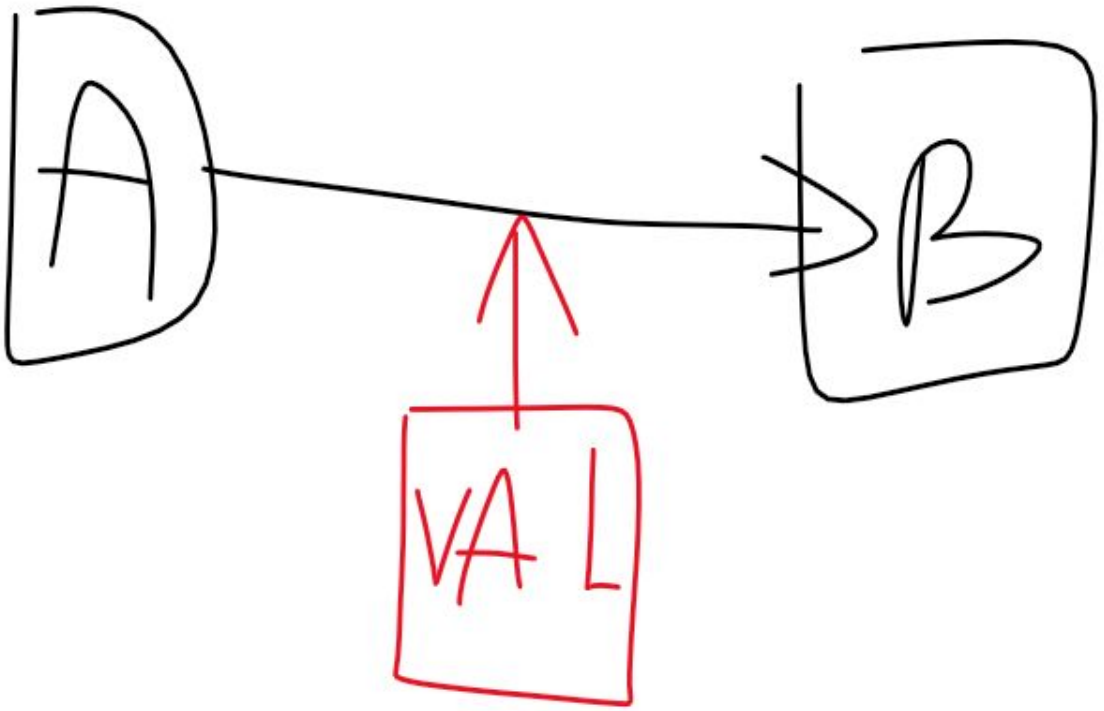
```
34  template<class T> class List
35  {
36      protected:
37          Link<T> *first;
38
39      public:
40          List():
41              first(nullptr)
42              {
43
44              }
45          List(const List &source); //konstruktor kopiujacy
46          virtual ~List();
47
48          virtual void add(T value);
49          virtual void deleteAll(); //delete all apart from head
50          T firstElement() const; //access first element
51          virtual bool includes(T value) const; //inclusion test
52          bool isEmpty() const;
53          virtual void removeFirst();
54
55          friend class Iterator<T>;
56  }
```

Klasa Link:

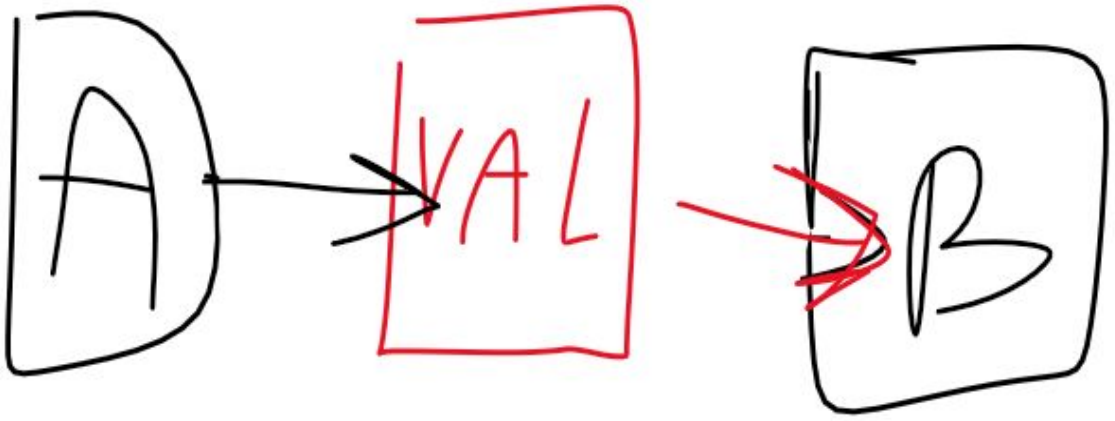
```
59  template<class T> class Link
60  {
61      private:
62          T value;
63          Link<T> *next;
64
65          Link(T val, Link *ptr): //dwa argumenty, wartość, oraz wskaźnik do kolejnego elementu
66              value(val), next(ptr)
67              {
68
69              }
70      public:
71          Link<T> *insert(T value); //funkcja jako wynik zwraca wskaźnik do nowego obiektu klasy Link
72
73          friend class List<T>;
74          friend vlass Iterator<T>;
75  }
```

funkcja **insert** wstawi element ZA SIEBIE (w kolejne miejsce listy) i zwróci nam jego adres.

A wstawia nowy element na listę, między A i B, a nowy element VAL ma wskazywać na B




```
78  template<classT> Link<T> *Link<T> :: insert(T value)
79  {
80      next = new Link<T>(value, next); //to next przyjmie wartość A, która wskazuje na B
81      return next; //teraz A wskazuje na nowy element
82  }
```



Definicja pozostałych funkcji:

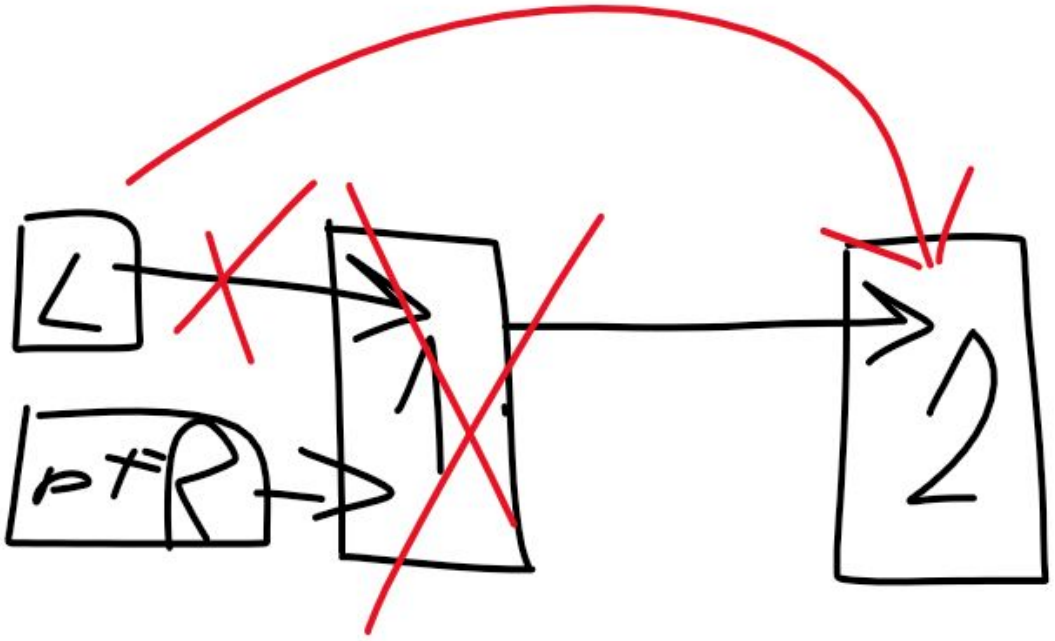
```
86  template <class T> T List <T> :: firstElement() const
87  {
88      assert(first != nullptr);
89      return first->value;
90  }
91
92  template <class T> bool List <T> :: isEmpty() const
93  {
94      return first == nullptr;
95  }
96
97  template <class T> bool List <T> :: includes(T val) const
98  {
99      for(Link <T> *p = first; p; p = p->next)
100     {
101         if(val == p->value)
102         {
103             return true;
104         }
105     }
106     return false;
```

for(Link <T> *p = first; p; p = p->next to pętla idąca krok po kroku po liście. Do tego lepiej przeładować operator **==** niż zadawać się na domyślne przeładowanie.

Usuwanie pierwszego elementu (kod):

```
110 template <class T> void List <T> :: removeFirst()
111 {
112     assert(first != nullptr);
113     Link <T> *ptr = first; //zapisuje nowy wskaźnik pokazujący na pierwszy element
114     first = ptr->next;
115     delete ptr;
116 }
```

Usuwanie pierwszego elementu (schemat):

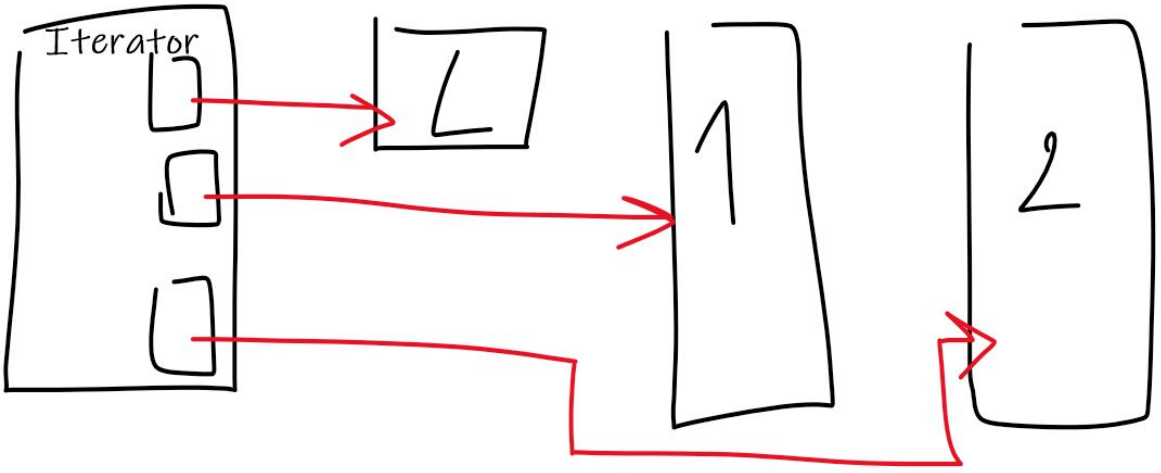


Usuwanie całej listy:

```
119  template <class T> void List <T> :: deleteAll()
120  {
121      Link <T> *next;
122      for(Link <T> *p = first; p; p = p->next)
123      {
124          next = p->next;
125          delete p;
126      }
127      first = nullptr;
```

Schemat identyczny co w usuwaniu pierwszego. Jak Indiana Jones, idziemy do przodu, odcinając za sobą.

ZŁOŻONE OPERACJE NA LIŚCIE JEDNOKIERUNKOWEJ



Definicja przykładowa iteratora:

```
131  template <class T> class Iterator
132  {
133      public:
134          Iterator(List <T> &aList);
135          virtual bool init();
136          virtual T operator()();
137          virtual bool operator !();
138          virtual bool operator ++(); //do przesuwania sie po liscie prefixowo, dla postfixu trzeba dodać(int)
139          virtual void operator =(T value);
140          void removeCurrent();
141          void addBefore(T newValue);
142          void addAfter(T newValue);
143
144      protected:
145          List <T> *myList; //wskaznik na liste, do ktorej nalezy iterator
146          Link <T> *previous; //poprzedni element listy wzgledem current
147          Link <T> *current; //nad ktorym elementem znajduje sie iterator
148  }
```

Składniki klasy Iterator:

```
151  template <class T> Iterator <T> :: Iterator(List <T> &aList):
152      myList(aList)
153      {
154          init();
155      }
156
157  template <class T> bool Iterator <T> :: init()
158  {
159      previous = nullptr;
160      current = myList -> first;
161      return current != nullptr;
162  }
163
164  template <class T> T Iterator <T> :: operator ()()
165  {
166      assert(current != nullptr);
167      return current -> value;
168  }
```

```
170  template <class T> void Iterator <T> :: operator =(T val)
171  {
172      assert(current != nullptr);
173      current -> value = val;
174  }
175
176  template <class T> void Iterator <T> :: removeCurrent()
177  {
178      assert(current != nullptr);
179      if(previous == nullptr)
180      {
181          myList -> first = current -> next;
182      }
183      else
184      {
185          previous -> next = current -> next;
186      }
187      delete current;
188      current = nullptr;
189  }
```

```
191  template <class T> bool Iterator :: operator ++()
192  {
193      if(current == nullptr)
194      {
195          if(previous == nullptr)
196          {
197              current = myList -> first;
198          }
199          else
200          {
201              current = previous -> next;
202          }
203      }
204      else
205      {
206          previous = current;
207          current = current -> next;
208      }
209      return current != nullptr;
210  } //valid only for prefix!!!!
```

```
213  template <class T> bool Iterator <T> :: operator !()
214  {
215      if(current == nullptr and previous != nullptr)
216      {
217          current = previous -> next;
218      }
219      return current != nullptr;
220  }
```

```
222  template <class T> void Iterator <T> :: addBefore(T val)
223  {
224      if(previous)
225      {
226          previous = previous -> insert(val);
227      }
228      else
229      {
230          myList -> List<T>::add(val);
231          previous = myList -> first;
232          current = previous -> next;
233      }
234  }
```

```
236  template <class T> void Iterator <T> :: addAfter(T val)
237  {
238      if(current)
239      {
240          current -> insert(val);
241          return;
242      }
243      if(previous)
244      {
245          current = previous -> insert(val);
246      }
247      else
248      {
249          myList -> List<T>::add(val);
250      }
251  }
```

LISTY POSORTOWANE

Obiekty listy są uporządkowane zgodnie z pewnym kryterium. Konieczna jest redefinicja operacji **add**.

```
254  template <class T> class SortedList: public List<T>
255  {
256      public:
257          virtual void add(T value);
258  };
259
260  template <class T> void SortedList<T> :: add(T value)
261  {
262      Iterator <T> itr(this);
263      for(itr.init(); ! itr; ++itr) //ustaw na poczatku listy; dopoki zwraca true; przejdz dalej
264      {
265          if(value < itr())
266          {
267              break;
268          }
269      }
270      itr.addBefore(value);
```

ALGORYTM SORTOWANIA PRZEZ WSTAWIANIE PRZY LISTACH

```
274  template <class T> void insertionSort (Vector<T> &v)
275  {
276      SortedList<T> sorter;
277      //wez cala tablice Vector i stworz z niej liste posortowana
278      VectorIterator<T> vitr(v); //zaprojektowac iterator Vector w domu
279      for(vitr.init(); ! vitr; ++vitr)
280      {
281          sorter.add(vitr());
282      }
283
284      //skopiuj z powrotem do tablicy
285      int i = 0;
286      Iterator<T> itr(sorter);
287      for(vitr.init(); ! vitr; ++vitr)
288      {
289          v[i++] = itr();
290      }
291  }
```

Złożoność obliczeniowa sortowania przez wstawianie:
$\frac{(n^2+n)}{2}$ porównań i $\frac{(n^2+n)}{2}$ przypisań.
Złożoność obliczeniowa przy użyciu listy:
$\frac{(n^2+n)}{2}$ porównań i n przypisań.
ALE JEST TO ŹLE NAPISANE, BO TWORZY SIĘ MEMORY LEAK! Usunie się tylko wskaźnik na pierwszy element, reszta listy zostanie jako cmentarzysko.

INNE TYPY LIST

- Lista cykliczna - jednokierunkowa, gdzie ostatni element wskazuje na pierwszy element.
- Lista liniowa dwukierunkowa - każdy element wskazuje na następny oraz poprzedni element
- Lista dwukierunkowa cykliczna - każdy element wskazuje na następny oraz poprzedni element, dodatkowo pierwszy wskazuje na ostatni element, ostatni wskazuje na pierwszy
- Lista z “sentinelem” - do głowy listy dodajemy wskaźnik, który wskazuje na ostatni element (może być jedno i dwukierunkowa, a także cykliczna)

NAZWY (NIE)ZALEŻNE

```
294  template <class T> class Base
295  {
296      public:
297          void baseFuncion()
298          {
299
300          }
301  };
302
303  template <class T> class Derived: public Base<T>
304  {
305      public:
306          void derivedFunction()
307          {
308              baseFunction();
309          }
310  }
311
312  //NIEWLASCIWE! OD 2011 ROKU JEST TO ZLY ZAPIS
```

Dziedziczenie przy szablonach bardzo krzywo działa i często ignorują wnętrzności szablonów. Oto dwa rozwiązania

```
313  template <class T> class Derived: public Base<T>
314  {
315      public:
316          void derivedFunction()
317          {
318              this -> baseFunction();
319              Base<T> :: baseFunction();
320          }
321  }
```

PRZYKŁAD - LISTA Z “SENTINELEM”

Pojawia się tu w głowie wskaźnik pokazujący na koniec listy. Trzeba będzie przeprojektować klasę List.

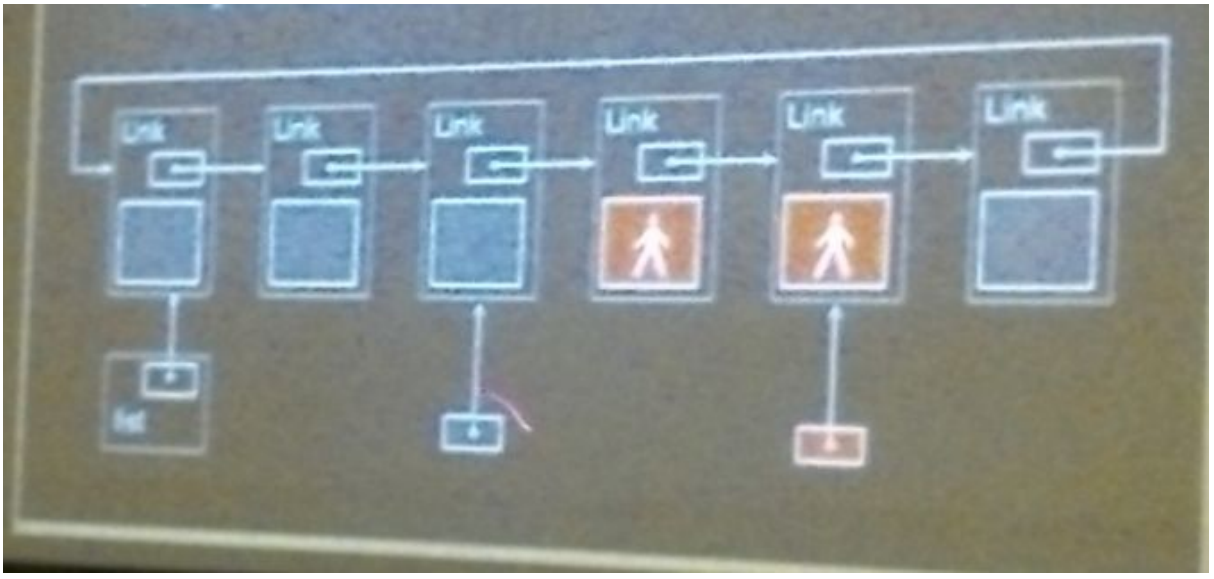
```
324  template <class T> class DoubleEnded: public List<T>
325  {
326      protected:
327          Link<T> *last;
328      public:
329          DubleEnded():
330              last(nullptr)
331              {
332
333              }
334          virtual void add(T value);
335          virtual void deleteAll();
336          virtual void removeFirst();
337          void addToEnd(T value);
338  };
```

```
340  template <class T> void DoubleEnded<T> :: add(T value)
341  {
342      if(this -> isEmpty())
343      {
344          List<T> :: add(value);
345          last = this -> first;
346      }
347      else
348      {
349          List<T> :: add(value);
350      }
351  }
```

```
353  template <class T> void DoubleEnded<T> :: addToEnd(T value)
354  {
355      if(last != nullptr)
356      {
357          last = last -> insert(value);
358      }
359      else
360      {
361          add(value);
362      }
363  }
```

```
365  template <class T> void DoubleEnded<T> :: deleteAll()
366  {
367      List<T> :: deleteAll();
368      last = nullptr;
369  }
```

KOLEJKA FIFO (FIRST IN FIRST OUT)
Naturalne rozwiązanie za pomocą listy ze wskaźnikiem na początek i koniec
Implementacja w formie bufora cyklicznego - bardziej efektywne zarządzanie pamięcią.



Znacznie bardziej oszczędne niż wstawianie i usuwanie elementów do listy.

KOLEJKA LIFO (STOS)
Prosta implementacja na bazie klasy List.
Operacja push (połóż na stosie) - funkcja add().
Operacja pop (pobierz ze stosu) - kombinacja firstElement() i removeFirst().

Notacja polska - zastosowanie stostu
Konwersja notacji wrostkowej (infiksowej) na beznawiasową notację przyrostkową
Notacja polska (prefiskowa) - 1924

- Jan Łukasiewicz (1878 - 1956)

Odwrotna notacja polska (postfiksowana) - 1954

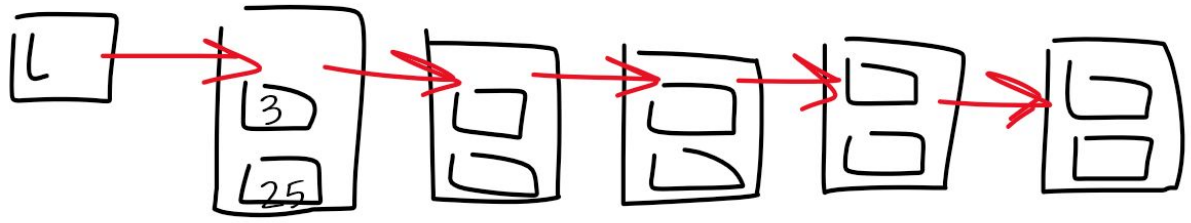
- Charles Hamblin (1922 - 1985)

Odwrotna notacja polska:
5 9 8 + 4 6 * * 7 + * = 5 (9 + 8) (4 * 6) * 7 + * = 5 ((9 + 8) * (4 * 6)) 7 + * = (5 * (((9 + 8) * (4 * 6)) + 7))

ARYTMETYKA WIELOMIANÓW
Środowisko umożliwiające reprezentację wielomianów posaci a_n*x^n
Podstawowe operacje - wyznaczanie sumy i iloczynu.
Porównywane i wydruk wielomianów
Realizacja za pomocą list - szczególnie efektywna dla wielomianów rzadkich (mała wartość stosunku liczby niezerowych składników do rzędu wielomianów).

Operacje na wielomianach zastąpione odpowiednimi operacjami na listach jednokierunkowych:

3x^25+8x^15+13x^8+27x^5+6



Definicja klasy:

```
class Term {
public:
   Term (int coef, int pow);          // constructor
   int  coefficient;                  // data fields
   const int exponent; // once created never changes
};
```

Operacje pojedynczych składników:

```
Term operator* (const Term & left, const Term & right){
   Term result (left.coefficient * right.coefficient,
                left.exponent + right.exponent);
   return result;
}

Term operator+ (const Term & left, const Term & right){
   assert(left.exponent == right.exponent);
   Term result (left.coefficient + right.coefficient,
                left.exponent);
   return result;
}

bool operator== (const Term & left, const Term & right){
   return ((left.coefficient == right.coefficient)
            and (left.exponent == right.exponent));
}
```

Klasa wielomianów:

```
class Polynomial {
private:
   List <Term> terms; // data field
public:
   Polynomial (); // empty polynomial
   Polynomial (const Polynomial & p); // copy constructor
   Polynomial (Term & t); // a single term polynomial
   Polynomial (int v); // vx^0 = v
   Polynomial (int a, int b); // ax + b
   void operator=  (const Polynomial & right);
   void operator+= (const Polynomial & right);
   void operator*= (const Term &);
   friend Polynomial operator+ (const Polynomial &, const Polynomial &);
   friend Polynomial operator* (const Polynomial &, const Polynomial &);
};
```

Przeładowanie operatorów:

OPERACJA DODAWANIA I PRZYPISANIA

```
void Polynomial :: operator+= (const Polynomial & right){
   Iterator <Term> itrL(terms);
   Iterator <Term> itrR(right.terms);
   for (itrL.init(),itrR.init(); !itrL and !itrR; ++itrL){
      while (!itrR and itrL().exponent < itrR().exponent){
          itrL.addBefore(itrR());
          ++itrR;
      }
      if (!itrR and itrL().exponent == itrR().exponent){
         itrL = itrL() + itrR();
         ++itrR;
      }
   }
   while (!itrR){
      itrL.addBefore(itrR()); ++itrR;
   }
}
```

DODAWANIE PEŁNE

```
Polynomial operator+ (const Polynomial & left, const Polynomial & right){
// duplicate left using the copy constructor
   Polynomial result(left);
// use already defined operator +=
   result += right;
   return result;
}
```

MNOŻENIE PEŁNE

```
Polynomial operator* (const Polynomial & left, const Polynomial & right) {
// start with empty polynomial
   Polynomial result;
   Iterator <Term> itr (right.terms);
   for (itr.init(); !itr; ++itr)
      result += left * itr();
   return result;
}
```

MNOŻENIE PRZEZ POJEDYNCZY SKŁADNIK

```
// first define multiplication and assignment operator
void Polynomial :: operator*= (const Term & right) {
   Iterator <Term> itr (terms);
   for (itr.init(); !itr; ++itr)
      itr = itr() * right;
}
// and then actual multiplication
Polynomial operator* (const Polynomial & left, const Term & right) {
// duplicate left using the copy constructor
   Polynomial result(left);
   result *= right;
   return result;
}
```

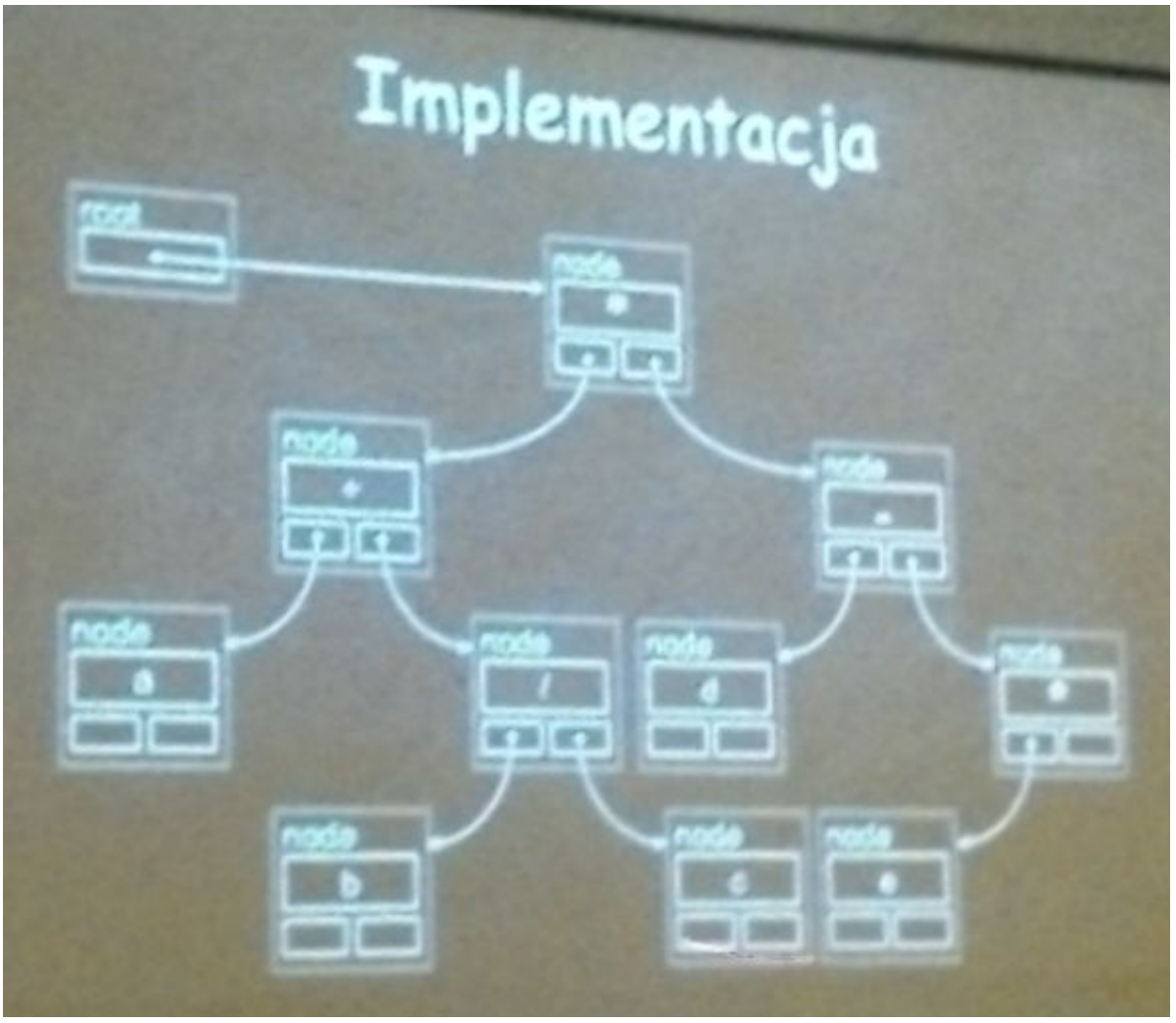
DRZEWA BINARNE

Drzewa binarne to wyjątkowy typ kopca.

DRZEWA BINARNE

```
template <class T>
class Node {
public:
   T value;
      .
protected:
   Node <T> *leftPtr;
   Node <T> *rightPtr;
};
```

Każdy węzeł (Note) naszego drzewa ma dwa wskaźniki - na prawy i lewy liść.

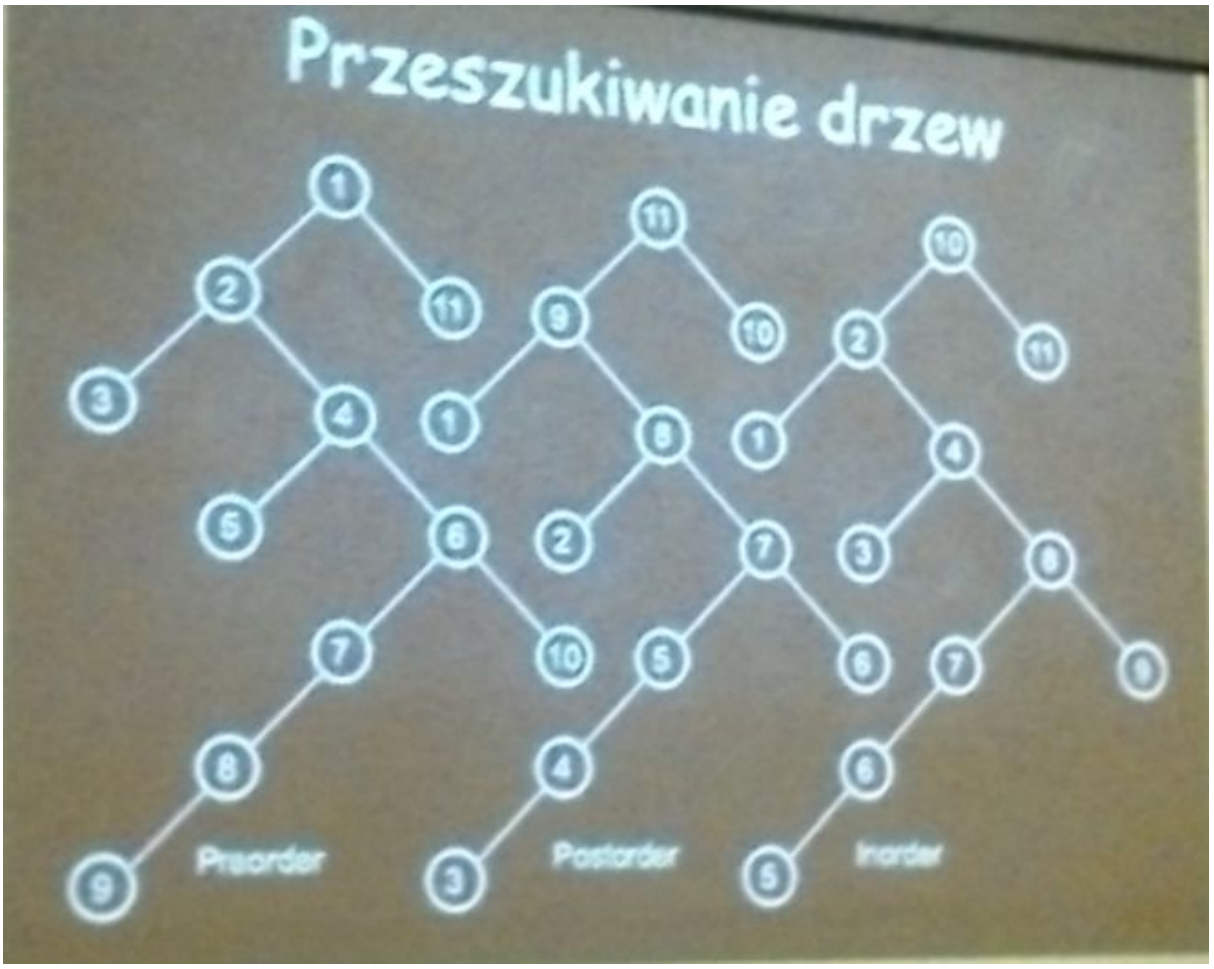


Drzewa binarne mają swoje własności:

- Liczba liści w pełnym drzewie binarnym o wysokości (liczbie poziomów) **h**: 2^(h-1)
- Liczba wierzchołków w pełnym drzewie o wysokości **h**: (2^h)-1
- Najdłuższa ścieżka w drzewie binarnym zawierającym **n** wierzchołków i całkowicie zapełnionym od lewej strony obejmuje log_2(n) warstw

Metody przeszukiwania drzew

- porządek poprzedzający (ang. preorder)
    - zbadaj wierzchołek bieżący
    - zbadaj lewe poddrzewo
    - zbadaj prawe poddrzewo
- porządek następujący (postorder)
    - zbadaj lewe poddrzewo
    - zbadaj prawe poddrzewo
    - zbadaj wierzchołek bieżący
- porządek wewnętrzny (inorder)
    - zbadaj lewe poddrzewo
    - zbadaj wierzchołek bieżący
    - zbadaj prawe poddrzewo



Funkcje rekurencyjne

```
template <class T> void Tree <T> :: preorder(Node <T> *ptr){
   if (ptr){
       examine (ptr); preorder (ptr->leftPtr);
       preorder (ptr->rightPtr);
   }
}
template <class T> void Tree <T> :: postorder(Node <T> *ptr){
   if (ptr){
       postorder (ptr->leftPtr); postorder (ptr->rightPtr);
       examine (ptr);
   }
}
```

```
template <class T> void Tree <T> :: inorder(Node <T> *ptr){
   if (ptr){
       inorder (ptr->leftPtr); examine (ptr);
       inorder (ptr->rightPtr);
   }
}
```

Zawsze wierzchołek V na lewo ma <V, na prawo >V
Najważniejsze operacje w drzewie:

- member(x) czy wartość X znajduje się w drzewie
- insert(x) wstaw nowy wierzchołek z wartością X
- delete(x) usuń wierzchołek z wartością X

MEMBER(X)

TEST PRZYNALEŻNOŚCI

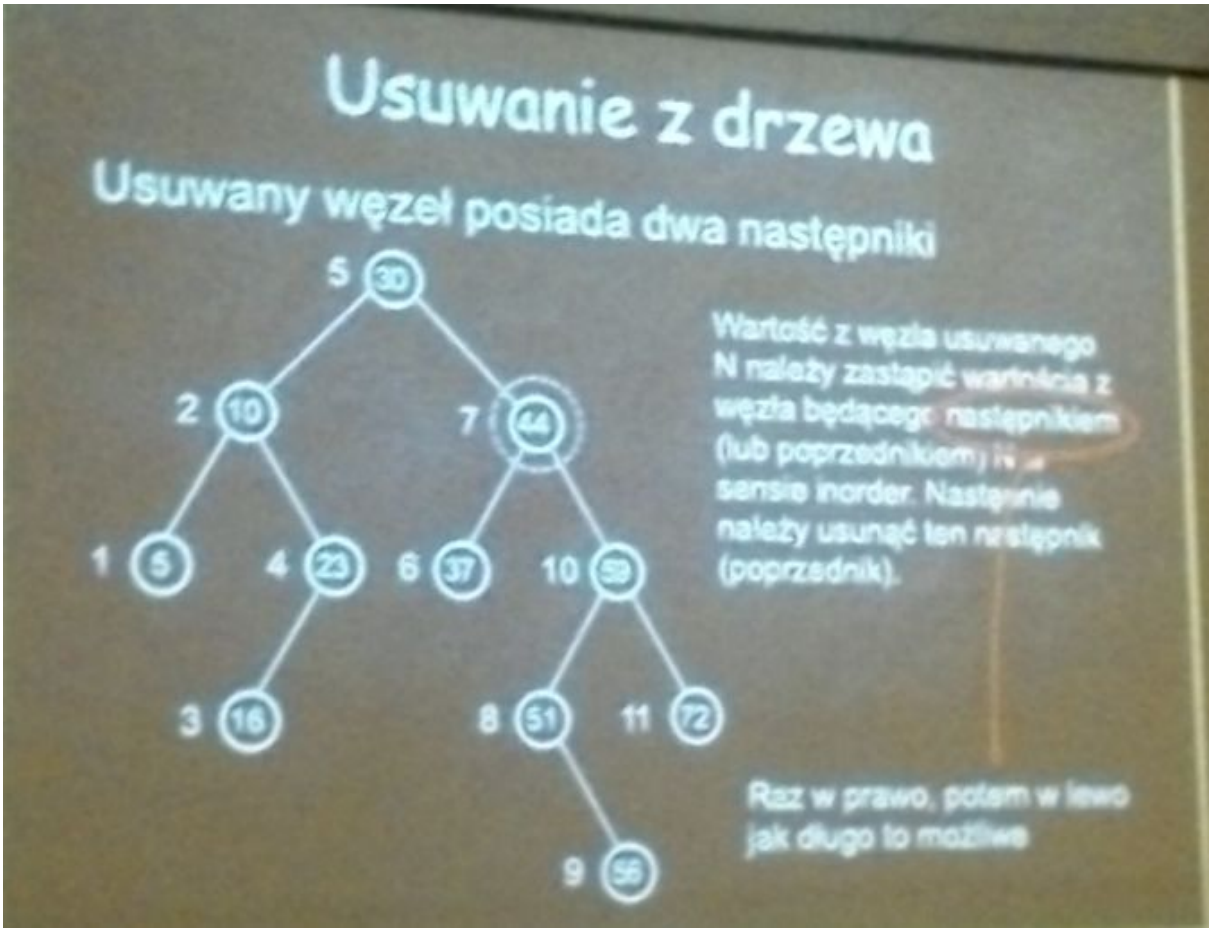
```
template <class T>
bool Tree <T> :: member(T val) const {
   Node <T> *current = root;
   while (current){
      if (current->value == val)return true;
      current = (val > current->value) ? current->right() : current->left();
   }
   return false;
}
```

INSERT(X)

WSTAWIANIE DO DRZEWA

```
template <class T>
void Tree <T> :: insert(T val) {
   if (root){
      Node <T> *current = root, *child;
      while (current){
         if (val < current->value){
            child = current->left();
            if (not child){
               current->left(new Node<T>(val));
               return;
            }
         }
         else {
            child = current->right();
            if (not child){
               current->right(new Node<T>(val));
               return;
            }
         }
         current = child;
      }
   }
   else root = new Node <T>(val);
}
```

DELETE(X)
Usuwanie z węzła, który ma dwa następniki jest trudne. Wartość z węzła usuwanego N należy zastąpić wartością z węzła będącego następnikiem lub poprzednikiem N w sensie inorder. Następnie należy usunąć ten następnik lub poprzednik.

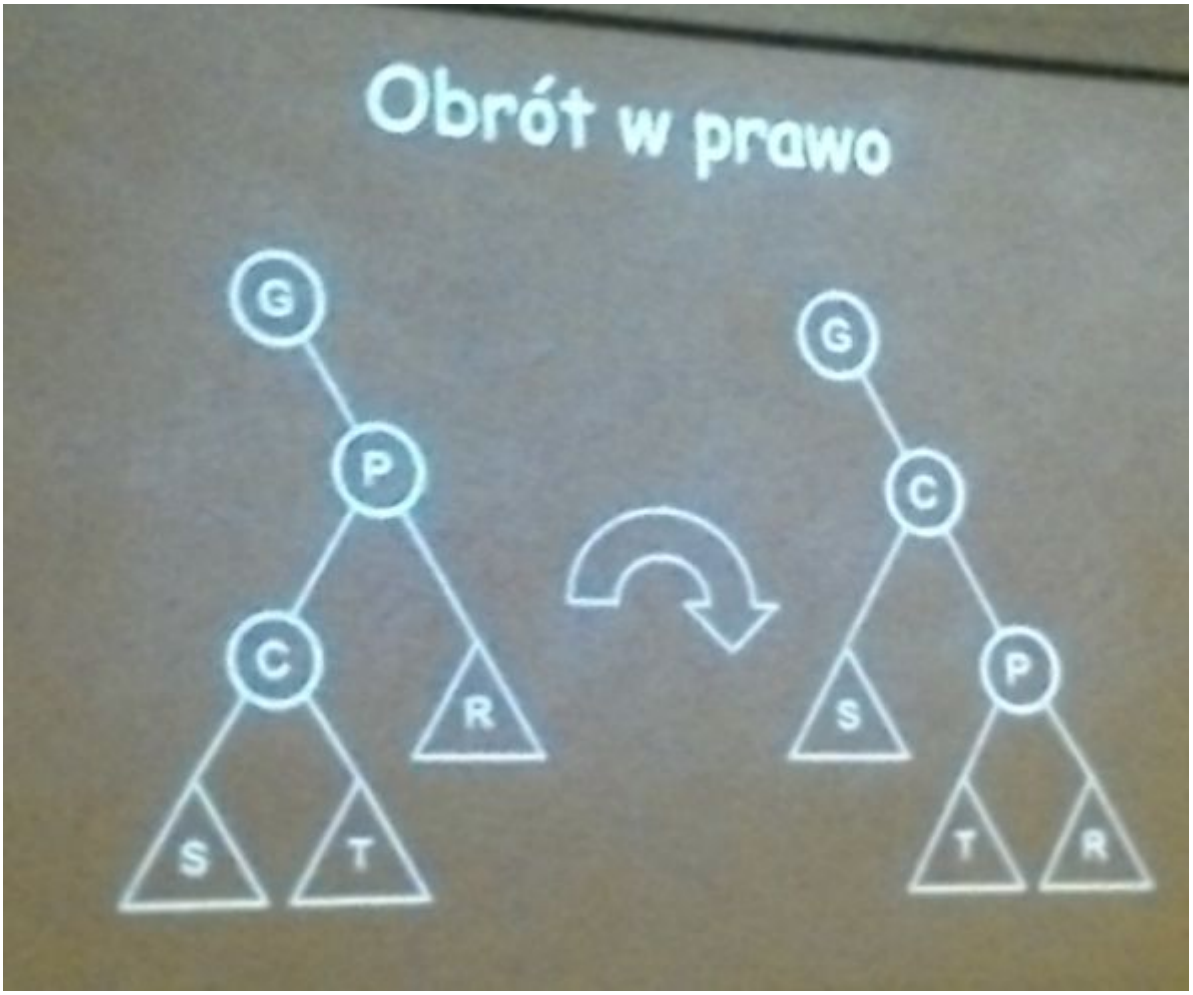


wstawiamy nr. 8 w miejsce nr 7 i usuwamy nr 8, co jest trywialne.

LOKALNE RÓWNOWAŻENIE DRZEWA
tak długo jak drzewo jest zrównoważone złożoność obliczeniowa jest logarytmiczna

- drzewa AVL (od nazwisk twórców Adelson-Velskii i Landis)
    - dla każdego wierzchołka w drzewie różnica między wysokościami jego poddrzew wynosi -1, 0 lub 1
    - dla każdego wierzchołka x współczynnik równowagi z(x)=hR(x) - hL(x) e{-1, 0, 1}

Podstawową operacją transformacji drzewa zaproponowaną przez AVL jest obrót węzła w drzewie. Obróćmy węzeł **P** w prawo.



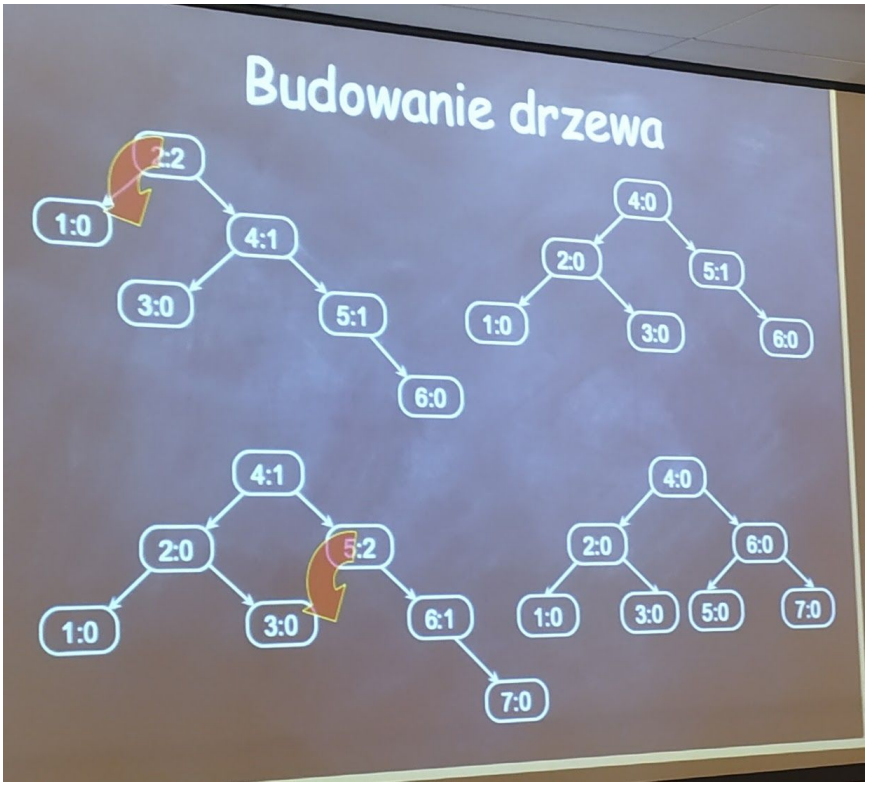
OBRÓT W PRAWO

```
template <class T>          // node A runs this function
AvlNode <T> * AvlNode <T> :: rotateRight(){
   AvlNode <T> *nodeB = left();
   // make reconnections
   left(nodeB->right());
   nodeB->right(this);
   // update balance factors
   int Abf = balFac, Bbf = nodeB->balFac;
   if (Bbf <= 0){
      if (Bbf > Abf)nodeB->balFac = Bbf + 1;
      else nodeB->balFac = Abf + 2;
      balFac = Abf - Bbf + 1;
   }
   else {
      if (Abf <= -1)nodeB->balFac = Bbf + 1;
      else nodeB->balFac = Abf + Bbf + 2;
      balFac = Abf + 1;
   }
   return nodeB;
}
```

Obracamy w prawo, **P** idzie na dół, **C** idzie do góry i elementy większe od **C**, a mniejsze od **P** podpinamy pod **P**

OBRÓT W LEWO

```
template <class T>          // node A runs this function
AvlNode <T> * AvlNode <T> :: rotateLeft(){
   AvlNode <T> *nodeB = right();
   // make reconnections
   right(nodeB->left());
   nodeB->left(this);
   // update balance factors
   int Abf = balFac, Bbf = nodeB->balFac;
   if (Bbf <= 0){
      if (Abf >= 1)nodeB->balFac = Bbf - 1;
      else nodeB->balFac = Abf + Bbf - 2;
      balFac = Abf - 1;
   }
   else {
      if (Abf <= Bbf)nodeB->balFac = Abf - 2;
      else nodeB->balFac = Bbf - 1;
      balFac = Abf - Bbf - 1;
   }
   return nodeB;
}
```



WSTAWIANIE NOWEGO ELEMENTU

```
template <class T>
AvlNode <T> * AvlNode <T> :: add(T val){
   if (val < value){ // insert into left subtree
      if (left()){
         int oldBf = left()-> balFac;
         left(left()->add(val));
         if((left()->balFac != oldBf) and left()->balFac)balFac--;
      }
      else {
         left(new AvlNode <T>(val));
         balFac--;
      }
   }
   else {
       if (right()){ // insert into right subtree
          int oldBf = right()-> balFac;
          right(right()->add(val));
          if((right()->balFac != oldBf) and right()->balFac)balFac++;
       }
       else {
          right(new AvlNode <T>(val));
          balFac++;
       }
   }
   if((balFac < -1) or (balFac > 1)) return balance();
   return this;
}

void AvlTree<T> :: add(T val){
   if(root)root = root->add(val);
   else root = new AvlNode <T>(val);
}
```

TEORIA GRAFÓW

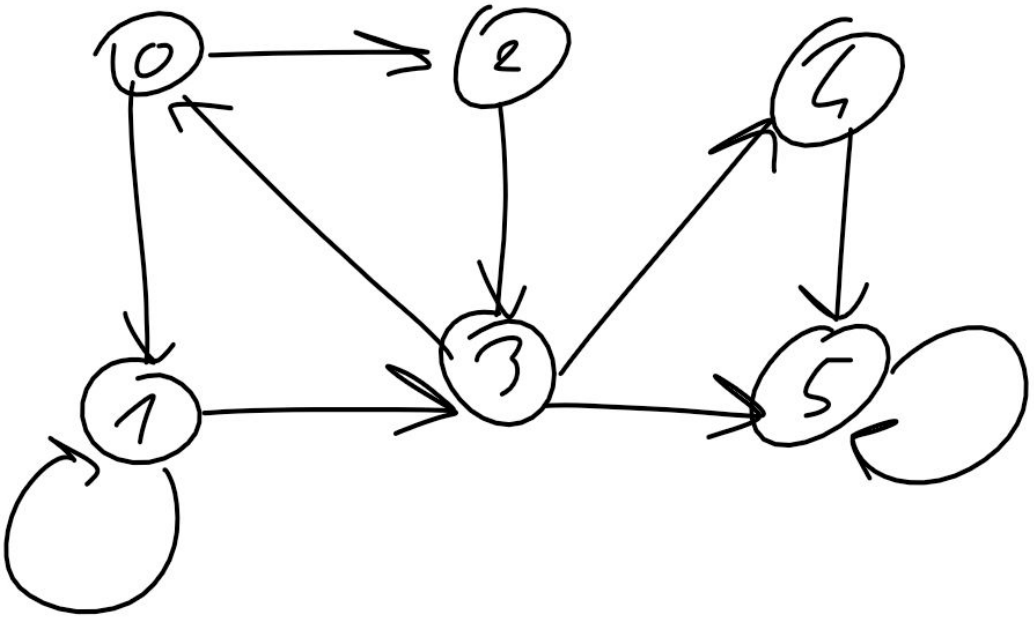
KOLOROWANIE MAPY

Każdą mapę można pokolorować minimalnie 4 kolorami. Autor tej tezy nie potrafił jej udowodnić.
Twierdzenie o kolorowaniu mpay (23.10.1852)
Dowód: Kenneth Appel i Wolfgang Haken, 1976
Każdy graf można pokolorować daną liczbą chromatyczną grafu. Graf reprezentujący mapę administracyjną mają liczbę chromatyczną 4.

Czy można pewne figury narysować bez odrywania ołówka i powtarzania fragmentów?

METODY REPREZENTACJI GRAFÓW

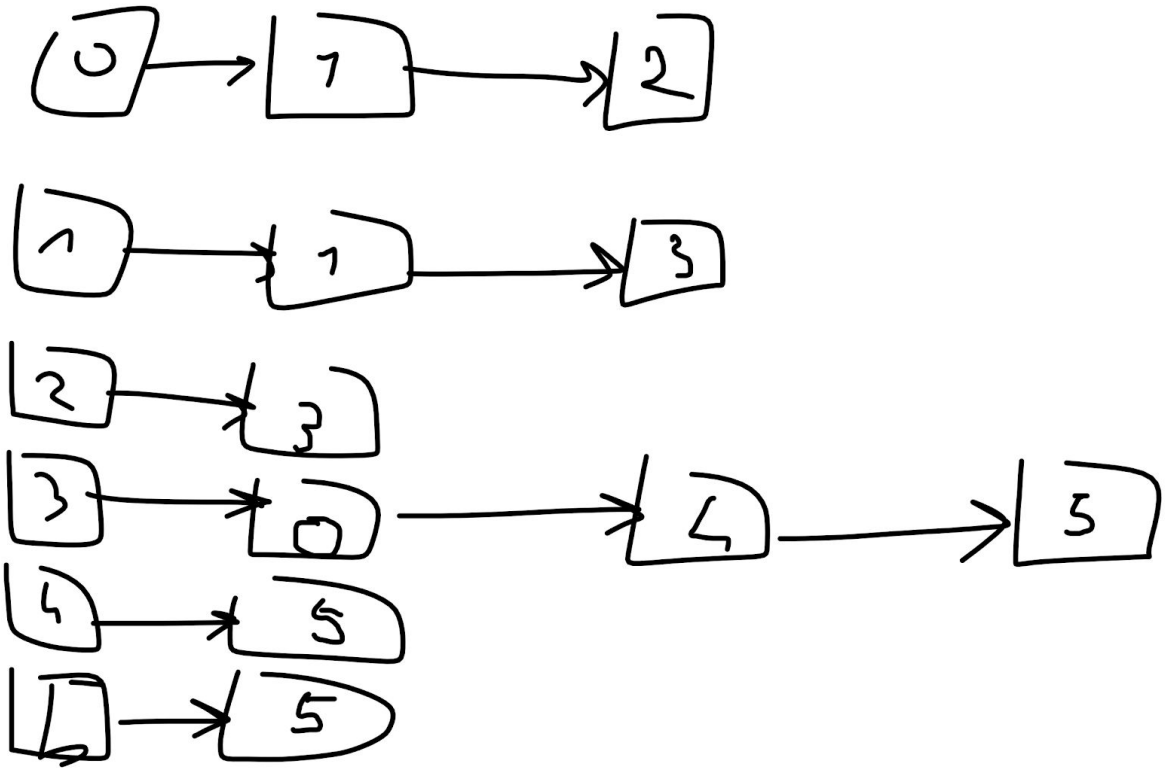
Grafy gęste (|E| bliskie |V|^2) - macierz połączeń



Tworzymy macierz o ilości wierszy i kolumn równej ilości elementów grafu. Tam gdzie jest połączenie jest 1 (np. między 0 i 1 jest połączenie, więc w wierszu 0, kolumnie 1 jest wstawione 1).

0	1	1	0	0	0
0	1	0	1	0	0
0	0	0	1	0	0
1	0	0	0	1	1
0	0	0	0	0	1
0	0	0	0	0	1

Listy krawędzi (incydencji)



METODY SPACEROWANIA PO GRAFIE

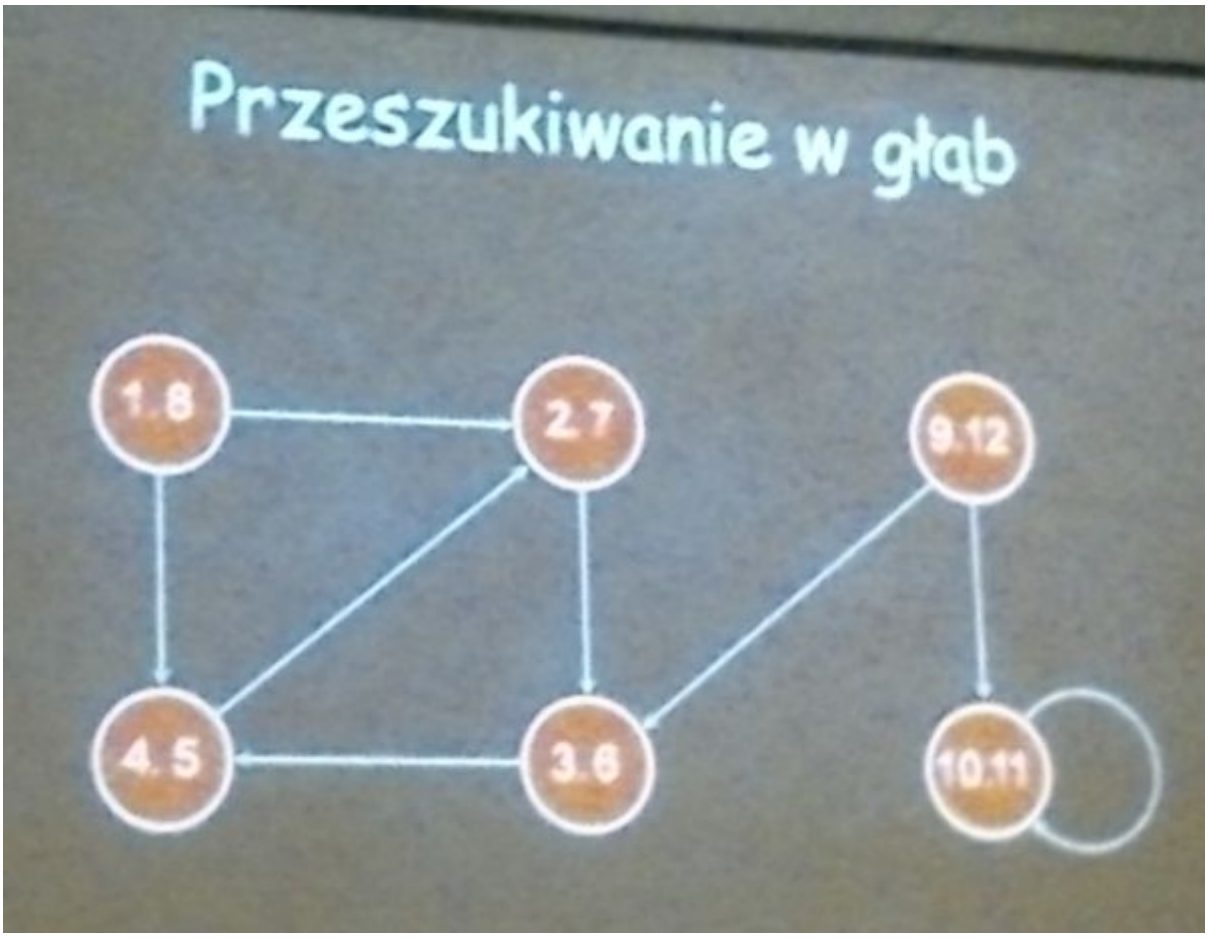
GRAFY – ALGORYTM PRZESZUKIWANIA W GŁĄB

```
void depthFirst (Node graph[], int current) {
   graph[current].color = green;      // it assumes all nodes
   graph[current].discovered = ++time;    // are initialized
   Vertex *ptr = graph[current].ptr;
   while (ptr){
      if (graph[ptr->id].color == black){
         graph[ptr->id].pred = current;
         depthFirst(graph, ptr->id);
      }
      ptr = ptr->next;
   }
   graph[current].color = red;
   graph[current].finished = ++time;
}

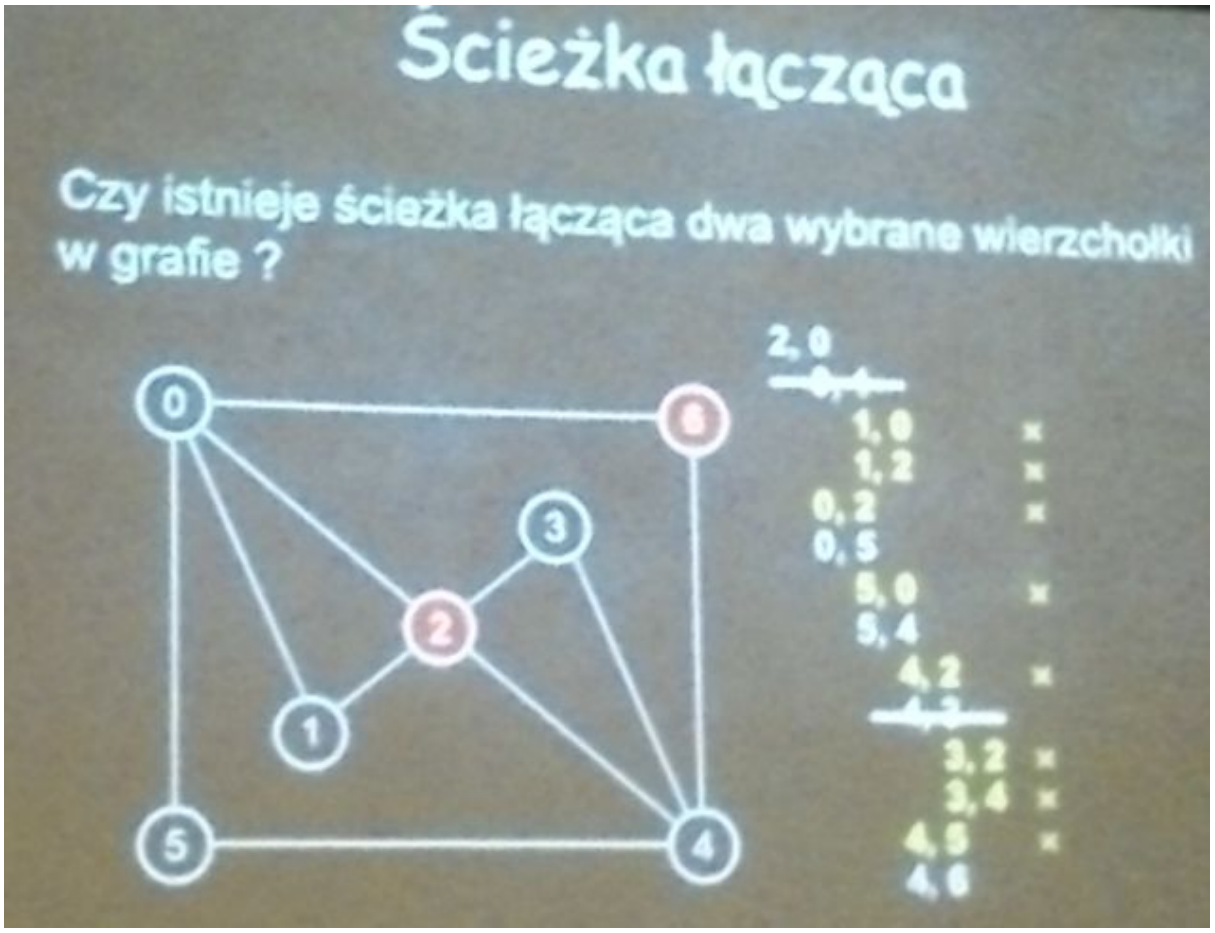
time = 0;
for (int v = 0; v < nodes; v++)
   if (myGraph[v].color == black)depthFirst(myGraph,v);
```

Startuję z węzła, oznaczam go, wybieram pierwszy, którego nie odwiedziłem, oznaczam go, następnie idę do pierwszego nie odwiedzonego, oznaczam. Jeśli już dojdę do końca (nie mam gdzie iść), robię krok w tył, szukam kolejnego możliwego, jeśli nie ma to znów krok w tył i tak aż skończą mi się opcje, aż zostanę na pierwszym węźle, z którego startowałem.

*“Dla danego wierzchołka **v** przeglądana jest jego lista incydencji. Wybierając krawędź **(v, u)** sprawdza się, czy wierzchołek **u** był już odwiedzany. Jeżeli tak, wybiera się następną krawędź. Jeżeli nie, algorytm przechodzi do wierzchołka **u** i postępuje jak wyżej.*
*Po wyczerpaniu listy incydencji wierzchołka **v** algorytm powraca do przeglądania listy incydencji wierzchołka, z którego osiągnął wierzchołek **v**.”*



ŚCIEŻKA ŁĄCZĄCA
Czy istnieje ścieżka łącząca dwa wybrane wierzchołki w grafie? Można to zrobić np. dzięki przeszukiwaniu wgłąb.

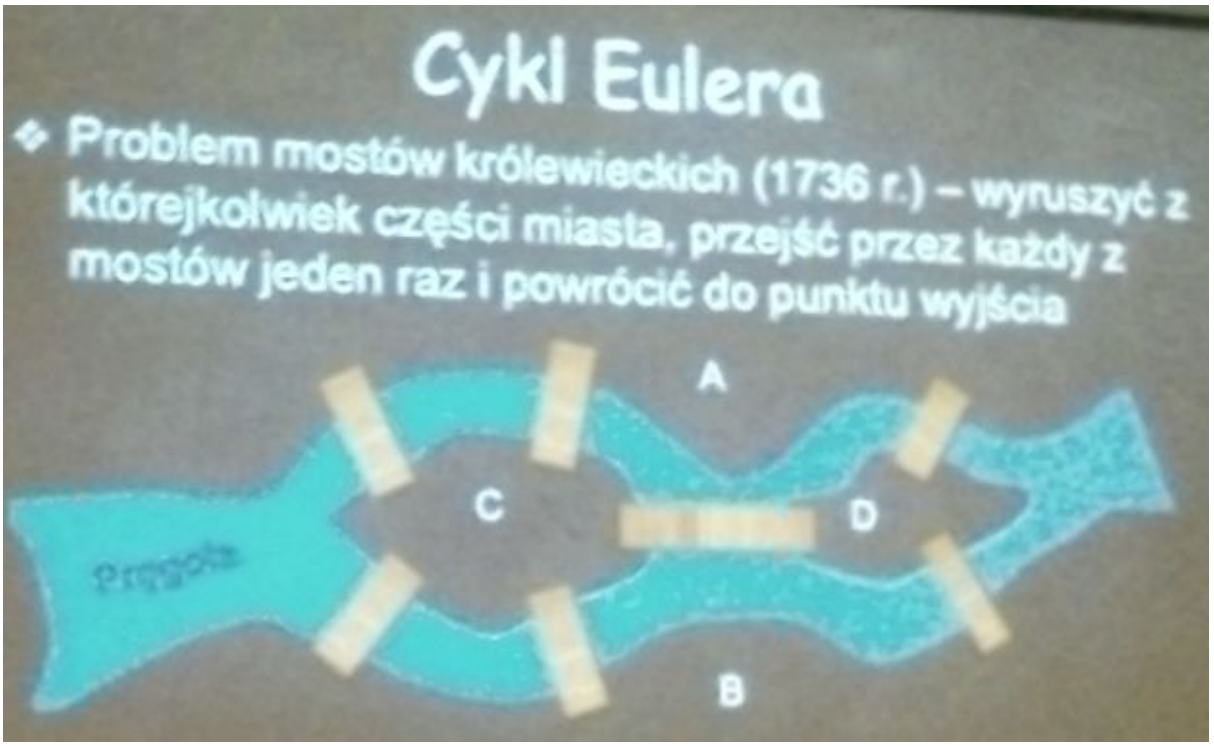


PROGRAM ZNAJDYWANIA ŚCIEŻKI

```
bool search (Node graph[], int v, int w) {
   if (v == w)return true;
   graph[v].visited = true;
   Vertex *ptr = graph[v].ptr;
   while (ptr){
      if (not graph[ptr->id].visited){
         if(search(graph, ptr->id, w)){
            cout << ptr->id << v;
            return true;
         }
      }
      ptr = ptr->next;
   }
   return false;
}

if(search(myGraph, v, w))cout << ”path found”;
```

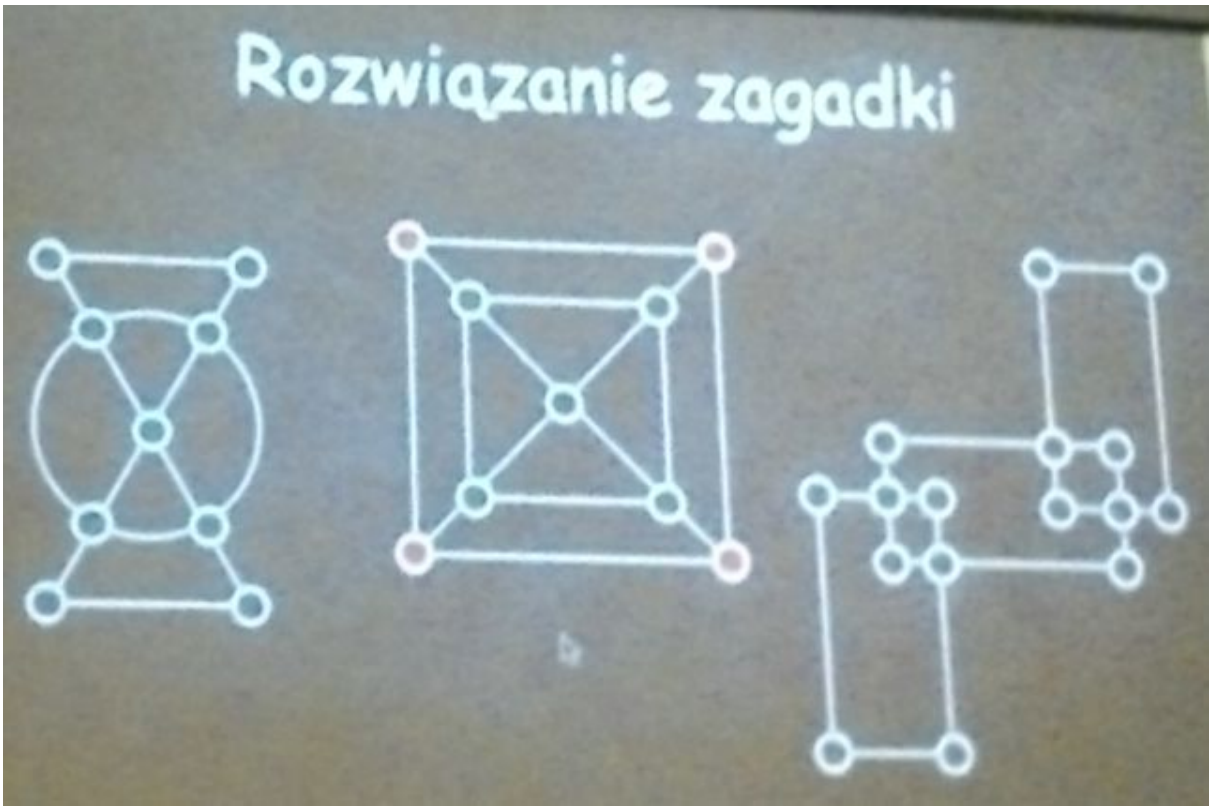
PROBLEM MOSTÓW KRÓLEWIECKICH



Czy w tym grafie istnieje ścieżka zamknięta, przechodząca przez każdą krawędź grafu dokładnie jeden raz.

Cykl Euler istnieje w grafie wtedy i tylko wtedy, gdy graf jest spójny i wszystkie jego wierzchołki są stopnia parzystego (mają parzystą liczbę krawędzi).
Ścieżka Eulera istnieje w grafie wtedy i tylko wtedy, gdy graf jest spójny i wszystkie jego wierzchołki są stopnia parzystego, z wyjątkiem dwóch grafów.
W grafie mostów Królewieckich nie ma ani ścieżki ani cyklu Eulera.

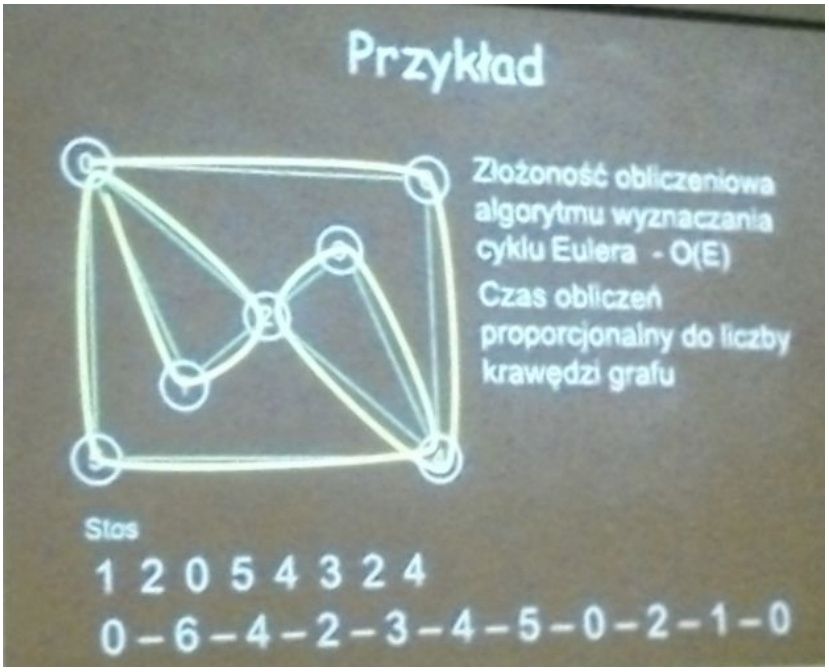
Problem z początku - trzeba wyznaczyć czy jest cykl i ścieżka Eulera.



Algorytm Hierholzer'a wyznaczania cyklu Eulera w dowolnym grafie - w sumie jeśli łazimy przeszukiwaniem wgłąb graf Eulerowski to jeśli dojdziemy znów do początkowego elementu grafu.

"*Wyznacz ścieżkę cykliczną **v-a-b-...-x-v**, umieszczając na stosie kolejne wierzchołki **a, b, ...** i usuwając z grafu odwiedzone krawędzie **v-a, a-b, …, x-v**.*

Powtórz poprzedni krok dla wierzchołków, które nie zostały całkowicie wyizolowane z grafu. Dodatkowe cykle są włączane w cykl wyznaczony wcześniej."



CYKL EULERA

```
int tour (Node graph[], int v) {
   while (true){          // program destroys graph
      Vertex *ptr = graph[v].ptr;
      if (ptr == nullptr)return v;
      int w = ptr->id;
      stack.push(w);
      removeEdge(v, w);  // delete two Graph objects
      removeEdge(w, v);
      v = w;
   }
}

// print Euler tour, if one exists
// the program stops, if a cycle is not found at any step
while (tour(myGraph, v) == v and not stack.empty()){
   v = stack.pop();
   cout << v << " ";
}
cout << endl;
```

CYKL HAMILTONA

Droga zamknięta, przechodząca przez każdy **wierzchołek** grafu dokładnie jeden raz.



ŚCIEŻKA HAMILTONA

```
bool searchH (Node graph[], int v, int w, int d) {
// only for tiny graphs !  First call with d = V
   if (v == w)return (d == 0);
   graph[v].visited = true;
   Vertex *ptr = graph[v].ptr;
   while (ptr){
      if (not graph[ptr->id].visited){
         if(searchH(graph, ptr->id, w, d - 1)){
            cout << ptr->id << v;
            return true;
         }
      }
      ptr = ptr->next;
   }
   graph[v].visited = false;
   return false;
}
```

Nie używać nigdy programu o złożoności O(n)=n!
Jeżeli graf ma |V|=15 i znalezienie ścieżki Hamiltona zajmuje 1 sekundę, to:
|V|=19 -> 1 dzień
|V|=21 -> ponad 1 rok
|V|=23 -> ponad 600 lat

PROBLEM KOMIWOJAŻERA

Komiwojażer odwiedza pewną liczbę miast. Znane są odległości między miastami (koszt/czas). W jakiej kolejności odwiedzić miasta, by być w każdym dokładnie jeden raz i przebyć jak najkrótszą drogę.

Jest to jeden z najintensywniej badanych problemów matematyki dyskretnej. Brak efektywnego algorytmu dla przypadku ogólnego.
Rekordy:

- Niemcy (15112 miast, 2001)
- Szwecja (24978 miast, 2004)
- układ scalony (859000 węzłów, 2006)

Droga łącząca **1 904 711 miasta i osady**: najlepszy znany wynik - **7 515 772 107 metrów** (2018).

Jak to ogarnąć?

1. Wybierz dowolną permutację miast
2. Dokonaj próbnej permutacji dwóch miast
3. Jeżeli zmiana skraca całkowitą długość drogi, to ją akceptuj; jeżeli wydłuża, to zmianę akceptuj z P zależnym od wydłużenia: im większe, tym mniejsze P
4. Kontynuuj zmniejszając stopniowo P
5. Zatrzymaj, jeżeli rozwiązanie nie zostało poprawione w pewnej liczbie kroków

Permutacja powodująca przyrost drogi o X jest akceptowana z $P=e^{-aX}$
a - stopniowo zwiększany parametr kontrolny
Nazywa się to **Symulowanego wyżarzania**.

METODA PRZESZUKIWANIA GRAFU WSZERZ

*Rozpoczynając od wybranego wierzchołka **v**, algorytm systematycznie znajduje wszystkie wierzchołki w odległości **s** od **v** przed zlokalizowaniem wierzchołków w odległości **s+1**.*
Znajdowanie najkrótszej ścieżki (najmniejsza liczba krawędzi) z wierzchołka **v** do każdego wierzchołka osiągalnego. Stosowana zarówno w skierowanych jak i nieskierowanych.

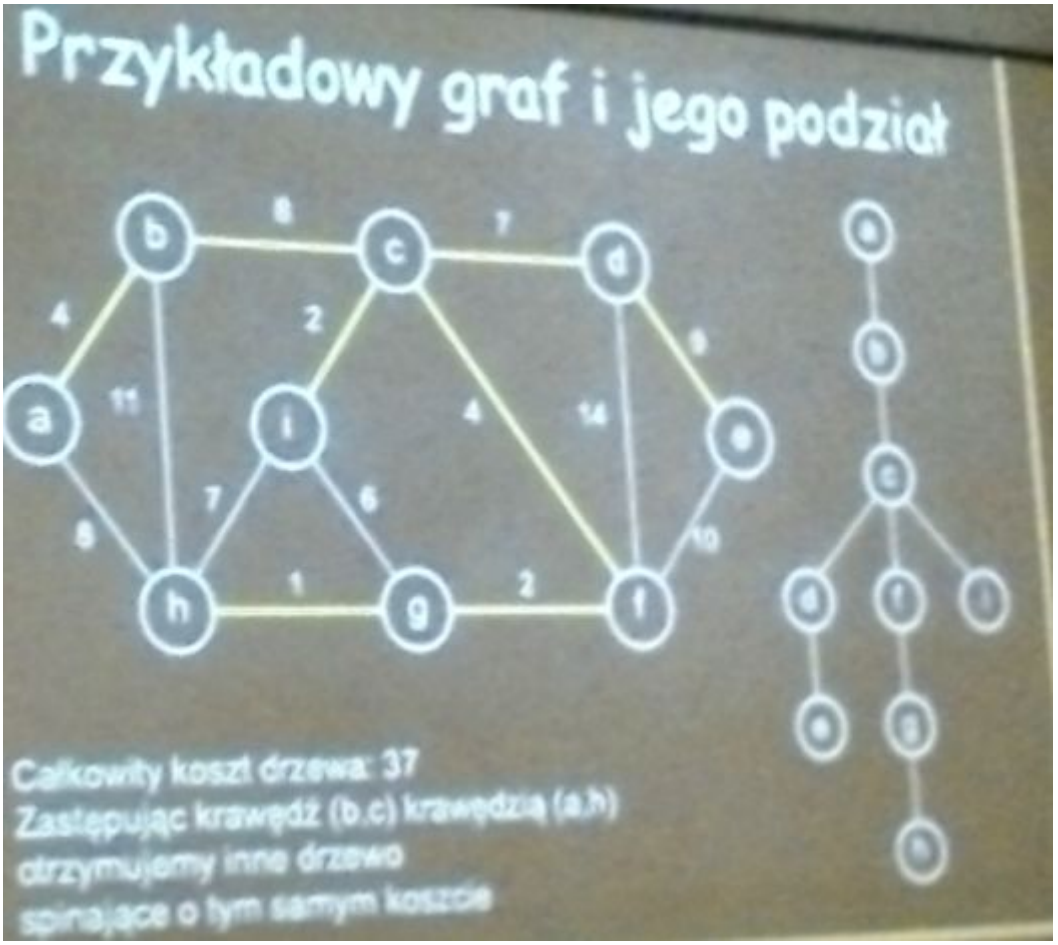
Drzewo osiągalności: najkrótsza ścieżka od danego wierzchołka do wierzchołka początkowego (poprzez usuwanie zbędnych ścieżek).

ALGORYTM PRZESZUKIWANIA WSZERZ

```
void breadthFirst (Node graph[], int start) {
   graph[start].color = green;       // it assumes all nodes
   graph[start].distance = 0;    // are properly initialized
   enqueue(queue,start);
   while (not empty(queue)){
      int v = dequeue(queue);  vertex *ptr = graph[v].ptr;
      while (ptr){
         if (graph[ptr->id].color == black){
            graph[ptr->id].color = green;
            graph[ptr->id].distance = graph[v].distance + 1;
            graph[ptr->id].pred = v;
            enqueue(queue,ptr->id);
         }
         ptr = ptr->next;
      }
      graph[v].color = red;
   }
}
```

DRZEWA SPINAJĄCE (SPANNING TREES) - DENDRYTY

Minimalne drzewo spinające - najmniejsza suma wag krawędzi. Algorytmy wyznaczające je są zachłanne (greedy algorithm)



Podział grafu - zbiór krawędzi, który łączy jedną grupę wierzchołków z drugą.

Algorytm Prima:

```
void primSpanningTree (Node graph[], int root) {
   Heap <Vkey> queue(nodes);
   for (int i = 0; i < nodes; i++){
       graph[i].tree = false;      // no member of tree
       queue.insert(Vkey(i, i == root ? 0 : INF));
   }
   while (not queue.empty()){
      Vkey u = queue.extractMin(); graph[u.id].tree = true;
      Vertex *ptr = graph[u.id].ptr;
      while (ptr){
         int v = ptr->id;
         if (not graph[v].tree and ptr->weight < key(v)){
            graph[v].pred = u.id;
            queue.updateKey(v, ptr->weight);
         }
         ptr = ptr->next;
      }
   }
}
```

W każdym kroku dodaj dodaj do drzewa “najlżejszą” krawędź łączącą wierzchołek z aktualnie istniejącego drzewa z wierzchołkiem jeszcze do drzewa nie należącym (wybierane krawędzie znajdują się na przecięciu grafu).
Algorytm jest zachłanny!
Złożoność tego algorytmu to
Inicjacja: $V \log_2 V$
Główna pętla: V
Pętla przeglądania list incydencji: $2E$
V - liczba wierzchołków, E - liczba krawędzi
Łączna złożoność: $E \log_2 V$

Algorytm Kruskala:
W kolejnych krokach rozłączne drzewa scalamy za pomocą krawędzi o minimalnym koszcie.
Początkowo V rozłącznych drzew jednoelementowych
Wynik: drzewo składające się z V-1 krawędzi
Szybkość algorytmu zależy od sposobu sortowania krawędzi oraz operacji na zbiorach.
Odrzuca się krawędzie zamykające drzewo w cykl.

Algorytm Boruvki
Dla każdego węzła wybierz krawędź o najmniejszym koszcie.